

★第5章

開発リソースは足りないのに増え続ける重圧

《登場人物》

湊太郎

田中部長

飛鳥さくら

山田健一

佐藤

高橋美咲

5-1

開発リソースが足りない中で
開発速度を上げる

ストーリー

新たな課題の顕在化

改善活動を始めて3カ月が経過した。湊のチームは確実に変化していた。

要件定義の事前レビュー会が導入され、仕様変更は週に3-5回から週に1-2回に減り、週1回のリファクタリング時間も確保されて技術的負債の返済が少しずつ進んでいる。朝会も定着し、チーム内のコミュニケーションは改善した。

でも、新たな課題が浮上していた。

金曜日の午後、田中部長から呼び出しがあった。部長室のドアをノックし、湊は中に入った。

「湊君、改善活動は良いけど、開発スピードは大丈夫？」

田中部長の表情には、経営陣からのプレッシャーが滲んでいた。

「確かに、仕様変更は減りましたし、影響調査もしやすくなりました。でも、丁寧にやると時間がかかる実感もあります」

湊は正直に答える。改善活動によって品質は向上したが、開発速度は以前と変わらないか、むしろ少し遅くなった気がする。

「経営戦略と競合分析を詰め直した場で、市場投入のスピードが急に焦点になった。経営陣からはもっと早く機能を出してほしいと言われている。改善は良いけど、スピードも大事なんだ」

田中部長の言葉に、湊は返答に困った。品質と速度、どちらも大事なのに、どう両立すればいいのか。口を開こうとしても、答えがまとまらない。

部長からの新たなプレッシャー

チームミーティングで、湊は状況を共有した。会議室の窓から午後の光が差し込み、メンバー全員が湊を見ている。

「部長から呼び出されて、開発スピードについて話がありました。改善活動で品質は向上したけど、速度は以前と変わらない。経営会議で『競合に後れを取っている。開発スピードを上げろ』と厳しく言われたそうです」

湊はそう言いながら、田中部長から聞いた話を詳しく説明した。

「競合A社が先月、勤怠管理にAI分析機能を追加した。我々も四半期以内に対応しないと、機能優位性を失うと部長は言っていました。それから、営業部長からは『機能が足りないと契約を失う。四半期で強豪と機能優位性として遜色がないように特に注力したい5個の施策をリリースしたい』と要望が来ている。マーケティング部からも『新機能のリリースがないと、プロモーション材料がない。四半期ごとに新機能を出さないと、市場での存在感が薄れる』と言われているそうです」

湊は少し間を置き、田中部長の言葉を続ける。

「部長は、経営陣からの圧力は理解できるが、開発組織として現実的にどう対応していくべきか、現有リソースでどうにかしてほしいというのが本音だと言っていました。人を増やす予算もないし、時間もない。でも、競合に後れを取るわけにはいかない。開発チームに何か手はないか、と相談されました」

山田が腕を組みながら口を開いた。

「人手を増やすしかないのかな？でも、採用には時間がかかるし、教育にも時間がかかる」

湊は部長から聞いた過去の経験を共有し、こう続けた。

「去年、エンジニアを2人採用したが、オンボーディングに3カ月かかり、すぐには戦力にならなかった。その間に競合に差をつけられた。採用しても、プロジェクトの文脈を理解するのに時間がかかり、期待した成果が出なかった。正社員を採用するにしても、リードタイムの長さや採用面談などの労力を考えると、四半期というタイムリミットには間に合わない」と部長が言っていました」

「予算的に人をこれ以上増やすのは難しいです。経営陣からは、現有リソースでどうにかしてほしいと言われています」

最近、開発チームのミーティングにも参加してくれるようになった飛鳥は資料に目を落とし、困った様子で答える。

「過去に業務委託でエンジニアをあまり深く考えずに採用したことがあるんですが、契約の関係もあり、あまり強いことは言えず逆に開発生産性がチームとして下がってしまい、結局やり直しになりました。業務委託での採用には慎重にならざるを得ないんです」

湊は黙って聞いていた。部長から、経営層には「開発が追いつかないなら人を増やせ」と言われる一方で、「何をいつまでに作るか」「そのために何人必要なのか」をすり合わせる時間が取れず、採用の人数目標だけが独り歩きしがちだという話も聞いていた。足りないのは、本当に人数なんだろうか。何か別のものが足りていないのかもしれない。そんな思いが頭をよぎった。

湊は深く考え込む。リソースは増やせない。でも、開発速度を上げる必要がある。どうすればいいのか。頭のなかで選択肢が巡るが、どれも決め手に欠ける。

「品質を保ちながら、これだけの施策を目標リリース日までにすべて出すスピード感は無理だと思う」

山田が腕を組み、率直に口を開いた。

「要望は理解できるけど、開発チームのキャパシティを考えると……」

飛鳥も不安そうだった。

「毎日残業が続いて、体力的にも限界です」

佐藤が俯くように小さな声で言う。チーム全体に疲労感が広がっていた。

山田が思い出したように口を開いた。

「そういえば、競合A社はAI活用で開発速度を50%向上させたと発表していたな。我々も対応しないと、取り残されている感がある」

AIエージェントにおける湊の疑問

湊は重圧を感じながらも、何か手はないかと考えていた。

その時、廊下を歩いていた湊に、他チームのエンジニアが声をかけた。

「湊さん、AIコーディング支援ツール使ってる？ コード書くの速くなるし、楽になるよ」

「AIコーディング支援ツール……ですか」

「うん。AIがある程度コードを自動生成してくれるから、開発速度が上がる。チーム全員がワークしだすとコードレビューが大変だけど生産量をたしかに増えた」

湊は興味を持った。もしAIツールで開発速度が上がるなら、本格的に導入してみる価値があるかもしれない。

その日の夜、湊は自宅のリビングで業務コードではなく趣味で書いているコードに対してAIコーディング支援ツールを試してみた。確かに簡単なプロンプトで指示を出すだけでコードを吐き出してくれて最新のモデルを使えばレビューで指摘する箇所も多くなかった。

でも、湊は違和感を覚えた。

生成コードへの違和感

生成されたコードを見ると、確かに動く。でも、コードの品質はどうなのか。保守性は？ 可読性は？ 技術的負債を増やすことにならないのか。

湊は生成されたコードをじっくり見直した。確かに動くが、無駄な処理があったり、命名が不適切だったり、コメントが不足していたりする。そのまま使うと、技術的負債が増える可能性がある。

湊は疑問に思った。速く書いても、後で直す時間が増えたら意味がない。自分の指示の出し方含めてまだ未熟かもしれないが、品質を犠牲にして速度を上げるのは、以前経験した失敗と同じではないか。業界ではAIで開発が速くなったという報告もあるが、条件やタスクによっては逆に遅くなるという研究も目にした。画面の生成コードを見つめながら、その違和感が消えなかった。

「これって本当に開発生産性向上できるのか？」

翌日のチームミーティングで、湊は疑問を共有した。

「AI コーディング支援ツールを試してみたんですが、確かにコードを書くスピードは速くなりました。でも、生成されたコードの品質に不安があります。そのまま使うと、技術的負債が増える可能性があると思うんです」

山田は腕を組んだ。

「コード補完ツールなら前に試したことがあるけど、コンテキストを理解しないまま適当なコードを吐くだけだった。技術的負債を増やす道具にしかならないと思ったよ」

湊は山田の反応に少し意外さを感じた。いつも冷静に新しい技術を評価する山田が、ここまで否定的なのは珍しい。

「ただ、湊君が言うように、生成されたコードをそのまま使わずにレビューして修正するなら話は別かもしれない。僕が昔試した時とは、ツールも進化しているだろうし。コード生成だけでなく、AIを技術的負債の検出に使ったり、テストケースの生成に使ったり、ドキュメント作成に使ったり。もっと戦略的に使えないだろうか？」

山田は少し考えてから、そう続けた。

山田の言葉に、湊は興味を持った。AIを「コードを早く書く道具」として使うのではなく、「**開発全体を効率化するパートナー**」として使う。それは新しい視点だった。

AI エージェントと人間の協働

その日の午後、湊は実際に業務でAIエージェントを試してみることにした。

湊はAIにある程度のシステムのコンテキストを与えたうえで「ユーザー登録機能を実装したい」と伝えた。AIは、認証チェック、バリデーション、データベースへの保存を含むコードを提案した。湊は、そのコードをレビューして、プロジェクトの規約に合わせて修正した。例えば、AIが提案したエラーハンドリングを、プロジェクトの標準的な方法に変更した。また、AIが提案した変数名を、プロジェクトの命名規則に合わせて修正した。

人間がコンテキストと目的を与え、AIがコードの骨組みを作り、人間が細部を調整する。その役割分担が、効率的だった。

湊は実感した。AIがコードを提案することで開発の初期段階が速くなる。でも、そのコードをそのまま使うのではなく、人間がレビューして修正することで品質も保てる。モデルの進化スピードによって日々進化があるが、速さと品質の両立がこの役割分担なら手の届くところにある気がした。

翌日のチームミーティングで、湊はペアプログラミングの成果を共有した。

「AIとペアプログラミングを試してみたんですが、先ほど話したような役割分担で実装速度が大幅に向上しました」

山田も同じように感じていた。

「その役割分担なら、開発速度も上がるし品質も保てる。現時点で理想的な使い方だと思う」

山田は続ける。

「すべてをAIにまかせてリリースまで完了させるのは、システムに責任を持っている側からするとまだ難しい。自分がある程度正解を知っているなかで、**いかに早くゴールに向かうことができるかの直列な速さ**に大きな効果を発揮しそうだと思う」

「でも、AIをうまく使うにはAIに何を伝えれば、良いコードが提案されるのか。それを学ぶ必要がある」

湊は頷きながら聞いていた。AIを戦略的に活用するには、AIの特性を理解し、適切な使い方を学ぶ必要がある。それは、新しいスキルだった。

「AIエージェントとの協働ワークで開発速度が大きく向上した。もっとAIを活用できないか。もしAIエージェントで開発量を確保できるなら、四半期というタイムリミットに対応できるかもしれない」

AIエージェントという新しい開発リソース

AIエージェント活用による初動の成功体験を踏まえて、湊はもっとAIを活用できないかと考えていた。その週末、湊は技術ブログを読んでいた。ある記事が目にとまった。

「AIエージェントによる開発量や生産量の確保が可能になり、人を採用するより予算的にもリードタイム的にはスケールの柔軟性がつきやすい」

湊は興味を持った。記事を読み進めると、AIエージェントを活用することで、開発量を確保できるという内容だった。

「これからは人の採用だけでなく、AIエージェントを活用することで開発量を確保する時代になる」

記事には、人を採用する場合とAIエージェントを導入する場合の比較が書かれていた。

- **予算** 人を1人採用するコスト（給与手当、福利厚生、採用費等）は年間800-1200万円。AIエージェントの月額料金は1-10万円程度。
- **リードタイム** 人の採用は3-6ヶ月かかる。AIエージェントの導入は1-2週間で完了。
- **教育コスト** 人の教育には数ヶ月かかる。AIエージェントの設定は数日で完了。
- **スケールの柔軟性** 人の採用は固定費になる。AIエージェントは需要に応じてスケール可能。

湊は考え込んだ。もしAIエージェントで開発量を確保できるなら、これ以上ないメリットだ。

チームでの検討

次の週のチームミーティングで、湊はAIエージェントの可能性を提案した。同じ会議室に、山田、飛鳥、佐藤らが顔を揃えている。

「週末、AIエージェントに関する記事を読みました。開発量や生産量の確保が可能になり、人を採用するより予算的にもリードタイム的にはスケールの柔軟性がつきやすいという内容でした」

飛鳥が目を輝かせ、前のめりになった。

「予算的にもリードタイム的にも柔軟性があるというのは、魅力的ですね。人を採

用する場合と比較するとどうですか？」

湊は記事の内容を共有した。

「人を1人採用する場合、年収+福利厚生で年間800-1200万円かかります。でも、AIエージェントは月額10-30万円程度です。年間で120-360万円程度なので、大幅にコストを削減できます」

「リードタイムも、人の採用は3-6ヶ月かかりますが、AIエージェントは1-2週間で導入できます。四半期というタイムリミットには間に合いそうです」

山田がうなずく。

「教育コストも、人の教育には数ヶ月かかりますが、AIエージェントの設定は数日で完了する。これは大きなメリットだね」

湊が腕を組んでから続ける。

「それに、今のような状況で急いで人を増やすのは逆効果になる可能性があります。前に読んだ『人月の神話』という本の中で遅れているプロジェクトに人員を追加すると、既存メンバーが新人の教育に時間を取られて、かえってプロジェクトが遅くなると書いてありました。今のチームは既に疲労感が広がっているのに、新人のオンボーディングに時間を割く余裕はないと思います」

山田が腕を組んで頷く。

「確かに、**ブルックスの法則**だね。急いで人を増やしても、教育コストがかかるだけでなく、既存メンバーの開発生産性も下がる。AIエージェントなら、その心配がない」

「それから、スケールの柔軟性も重要です。人の採用は固定費になりますが、AIエージェントは需要に応じてスケール可能です。忙しい時期だけ増やすこともできます」

湊が続ける。

「そして、AIエージェントはただ私たちと一緒にコードを書くだけではなく、独立的な動きができる存在を作ることができます」

「例えば、ポストモーテムの作成は、毎回時間がかかっていました。これを障害対応のチャットから自動で生成してくれるAIエージェントに任せることで、その時間を新機能開発に回せます」

山田も同意した。

「それから、ポストモーテムは比較的タスクが明確で、自動化しやすいです。最初のAIエージェントとしては適切だと思う」

翌日、湊はAIエージェントが生成したポストモーテムを確認した。品質は人間が作成したものと遜色なく、作成時間は大幅に短縮されている。

「これは使えますね。ポストモーテム作成にかかっていた時間を、新機能開発に回せます」

湊は満足そうだった。

「でも、AIエージェントが生成した内容を人間がレビューして、必要に応じて修正する必要がある。そうしないとチーム外の人から内容を聞かれたときに説明できないのは危険だ。**AIが書いたからわかりませんは通用しないからね**」

山田が注意点を指摘した。

湊は静かに頷いた。AIエージェントは魔法の杖ではない。でも、適切に使えば開発量を確保する有効な手段になるかもしれない。その手応えを、湊は胸に感じていた。

解説 AIを「コードを早く書く道具」以上のパートナーに

この節の論点：AIをコード生成だけの道具ではなく戦略的パートナーとして捉え直し、ペアプログラミングやAIエージェントとの協働まで視野に入れる必要性です。

⑤ ストーリーで描かれる「重圧」

なぜ「開発リソースは足りないのに増え続ける」が重圧になるのか。何が起きているかが関係者で共有されていないとき、その板挟みは構造的な課題としてのしかかります。湊のチームが直面したのも同じです。

- **経営陣・競合・営業・マーケからの圧力** 「開発スピードを上げろ」「四半期で複数の新機能や重点施策を」という要求。改善で品質は上がったが開発速度は以前と変わらない
- **人を増やせない制約** 予算制約、過去の採用失敗、オンボーディングに時間がかかり四半期のタイムリミットに間に合わない。現有リソースでどうにかしてほしいと言われる
- **「速く書いても後で直す時間が増えたら意味がない」** コード生成ツールで書くスピードは上がるが、本質的な開発生産性向上につながるかという疑問
- **AIをコード生成だけに使う限界** コードを早く書く道具として使うだけでは、技術的負債の増加や理解不足を招く可能性に気づき始めている

この重圧の背景には、**リソース不足と速度要求の板挟みのなかで、コード生成だけでは本質的な開発生産性向上にならない構造**があります。AIを戦略的パートナーとして捉え直す必要性に直面しているのです。

⑤ なぜコード生成だけでは不十分なのか

湊が気づいたように、AIツールを「コードを早く書く道具」として使うだけでは、本質的な開発生産性向上にはつながりません。

コード生成ツールを使うと、確かにコードを書くスピードは上がります。ただし効果はタスク種別・経験・文脈・検証コストに強く依存し、「速くなる」と一概には言えません。

この分野での実証研究の結果はまだバラつきがあります。

デミレルほか（Microsoft・Accenture・Fortune 100企業、計4,800名超の企業内ランダム化実験）では、Copilot使用でタスク完了数が約26%増、ジュニアで最大40%近い向上、シニアは7～16%に留まると報告されています。ペンほか（Copilot初の大規模RCT（無作為化比較試験））では、タスク完了速度が55.8%向上（信頼区間21～89%）という結果もあります。一方、METRの実験（ベテランOSS（オープンソースソフトウェア）開発者16名・246タスク、Cursor＋Claude等）では、AI使用群が平均19%遅く、参加者や専門家は速くなると予測していたにもかかわらず逆の結果でした。主観と実際のパフォーマンスの乖離（認知の歪み）が示された格好です。Nature Human Behaviourのメタ分析（106実験・370効果量、2020-2023）では、人間＋AIは平均として「最良の単独（人間またはAI）」には劣るが人間単独よりは良い（augmentationは成立、synergyは平均では成立しない）とされ、意思決定タスクでは損失、コンテンツ生成では利益が大きいとされています。評価では、測定条件（タスク・母集団・工程範囲）の一致を最優先すべきだという指摘があります。

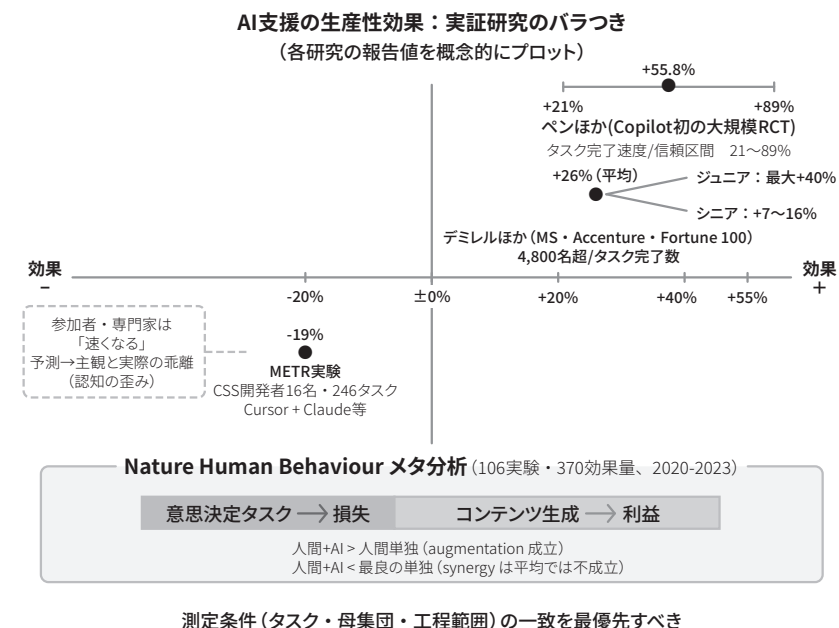


図 5.1：AI 支援の開発生産性効果：実証研究のバラつき（各研究の報告値を概念的にプロット）

さらに以下の問題が発生する可能性があります。

- **品質の低下** 生成されたコードが無駄な処理を含んでいたり、命名が不適切だったりする
- **技術的負債の増加** そのまま使うと、コードの複雑度が増し、将来的な開発速度が低下します。GitClear の分析（1.5～2 億行規模のコードベース、「Coding on Copilot」2024／「AI Copilot Code Quality」2025）では、AI 導入後に Churn（書いてすぐ消される行）が倍増し、リファクタリングの減少、コピー・クローンの急増（重複ブロックは2年前比で約10倍等）が報告されています。短期的なデリバリー速度は上がる一方、中長期的な技術負債を加速させる可能性が指摘されています
- **理解不足** 生成されたコードを理解せずに使うと、バグの原因がわからなくなる

⑤ AI は「増幅器 (Amplifier)」であり「鏡 (Mirror)」である

DORA (DevOps Research and Assessment) の 2025 年版レポート『State of AI-Assisted Software Development』は、AI を **増幅器 (Amplifier)** と捉える立場を示しています。生成 AI は組織の実践やプロセス、文化に埋め込まれた強みも弱みも同時に増幅します。開発と運用の流れが整っているチームでは実装の速さがレビューやテスト、リリースと噛み合い、成果が積み上がりやすい一方、レビュー待ちや手戻りが大きい組織では生成物の量や変更の速さだけが先に増え、負荷や不具合が表に出やすい、という関係が指摘されています。

レポートは AI 支援をツール導入の延長ではなく、組織の仕事の仕組みそのものを変える必要があると述べています。コードを書く入口だけが広がり、レビューやフィードバック、デプロイ、運用がボトルネックのままなら全体のスループットは伸びにくい、という論点です。AI 採用でコード生成は速くなる一方、バッチサイズが膨らみ小さな変更を頻繁に出すという DevOps の原則が崩れ、レビュー負荷の増大やデプロイの不安定化につながる例が報告されています。自動テストや成熟したバージョン管理、速いフィードバックといった制御システムが整っていれば、変更量の増加を受け止めて品質と安定性を保ちやすい、という対比です。

増幅という語は、組織の状態がそのまま結果に見えやすい、という読み方もできます。DORA 2025 は社内プラットフォームの品質、ワークフローの明確さ、チーム間の連携といった基盤が、AI の便益を組織の成果に結びつける条件だと述べています。ここではその含意を **鏡 (Mirror)** と呼びます。ツールの性能より、計測やレビュー、テスト、リリース、運用まで含めた実務の設計と文化が、活用の差をつけます。ボトルネックの移動やレビュー負荷の増大といった帰結は、本章の後半でもう一度整理します。

⑤ 人が足りないと言われたときに問うべきこと

「開発が追いつかない」「エンジニアが足りない」と言われたとき、解決策を「採用の強化」に求める前に、**本当に足りないのは人数なのか**を問い直すことが大切です。開發生産性はもはや「人数」の単純な関数ではなく、AI によるコーディング支援の普及で実装のボトルネック性は下がっています。つまり、かつての「人数＝アウトプット」という関係式は成立しにくくなっています。

検証すべき3つの軸は次のとおりです。

1. **戦略** 何を、いつまでに作るのか。プロダクトラインナップはどう拡大するのか。その計画は、誰が、どのような根拠で作ったのか。
2. **技術環境** AI の活用で、開發生産性はどこまで上がりうるのか。1 年前と同じ前提で人数計画を立てていないか。実装以外のボトルネック（要件定義・優先順位を決める力）はないか。
3. **組織能力** 要件を定義し、優先順位をつけ、「作らない」と決められる人がいるか。技術責任者の負荷は適切に分散されているか。経営陣の間で期待値はすり合っているか。

この3つが噛み合っていないまま人数目標だけが独り歩きしていると、採用しても「忙しいのに進まない」状態に陥りがちです。足りないのは人数ではなく「**何を作るか**」を**決める力**や**知恵**かもしれません。人を増やす前に、何が本当のボトルネックなのかを見極めることが、採用計画を立てる前に問うべき最初の問いです。章末の「人が足りないと言われたときの検証チェックリスト」も参照してください。

ブルックスの法則と急激な人員増加のリスク

フレデリック・P・ブルックスの『人月の神話』で知られる**ブルックスの法則**は、遅延しているソフトウェア・プロジェクトに人員を足しても納期が早まるとは限らず、むしろ遅れることがある、という観察です。新しく加わるメンバーはすぐにフル稼働に至らず、仕様・設計・コードベースの説明を受ける時間が必要です。既存メンバーは教育やレビューに時間を割かれ、もともとの実装や障害対応が後ろに回ります。人数が増えるほどコミュニケーションの通り道は増え、作業の再分割や統合のやり直しも発生します。順序に依存するタスクは、人月を増やしても線形に短くならない、という側面があります。

急いで人を増やすと、採用・受け入れ・オンボーディングの負荷が一時的に集中し、締切が重いチームでは新人を増やすほど、目の前の遅延を埋める時間が減る、という逆転が起きやすいです。疲労が広がっているときは、暗黙知の共有やメンタリングも滞ります。ストーリーで湊が『人月の神話』に触れたときの懸念は、こうした条件が揃うと、人員追加が短期的な成果を上げにくい、という点にあります。

このため、期日に合わせて開発量を確保したい場面で、人間の採用だけに頼るのではなく、オンボーディングに既存メンバーの稼働を大きく奪わない手段としてAIエージェントを検討する議論が出てきます。ただしAIはレビューや統制を不要にするものではなく、道具を足すだけで遅延が解消するわけではありません。

5-2 AI 活用の実践と 組織的課題

ストーリー

AI 活用方法のブレインストーミング

チームミーティングで、湊はAI活用について提案した。いつもの会議室に、5名のチーム全員が集まっている。

「AIをもっと戦略的に使えないだろうか？ コード生成以外の用途を考えてみませんか？」

メンバーがAI活用のアイデアを出し合った。

「技術的負債の検出はどう？ AIにコードベースを分析させて、問題箇所を特定してもらおう」

山田がホワイトボードに目をやり、最初に提案した。

「テストケースの自動生成もできるかもしれない。仕様書を読ませて、網羅的なテストケースを提案してもらおう」

高橋が画面を指さしてから続ける。

「ドキュメント作成支援も。コードを読ませて、自動的にドキュメントを生成する」

佐藤がホワイトボードを見上げて提案した。

「コードレビュー支援も。AIにコードを分析させて、潜在的な問題を指摘しても

らう」

別のメンバーが加わった。

ホワイトボードには、AI 活用のアイデアが並んでいた。

- 技術的負債の検出
- テストケース自動生成
- ドキュメント作成支援
- コードレビュー支援
- バグの予測
- パフォーマンス分析

湊はホワイトボードのアイデアを見つめながら言う。

「じゃあ、まずはテストケース自動生成から始めてみませんか？ 高橋さんと一緒に実験してみたいんです」

「面白い、実験してみよう」

山田が少し考えてから頷く。

リーダーとして湊が実験の方向性を整理した。まずはテストケース自動生成から始め、効果が確認できたら他の用途にも展開する。段階的に進めることで、リスクを最小限に抑えられる。

テスト自動化での AI 活用

翌週、高橋と湊はテストケース生成の実験を始めた。開発スペースの一角で、二人は仕様書と AI の画面を前に並んでいた。

高橋は仕様書を AI に読み込ませた。ユーザー登録機能、ログイン機能、データ分析機能。それぞれの仕様を AI に理解させ、テストケースを提案してもらう。

AI は数分でテストケースを返してきた。

「正常系のテストケース：ユーザー名とパスワードを正しく入力した場合、ログインに成功する」

「異常系のテストケース：パスワードが間違っている場合、エラーメッセージが表示される」

「境界値テスト：ユーザー名が 255 文字を超える場合、エラーメッセージが表示される」

高橋は息を呑んだ。

「これまで見落としていたエッジケースも含まれています。人間が考えると、正常系と基本的な異常系しか思い浮かぶことが多いんです。でも、AI は境界値テストやエッジケースも提案してくれる」

高橋は興奮しながら続ける。

「それから、テストケースの作成時間も大幅に短縮できました。以前は 1 つの機能につき、テストケースを作成するのに半日かかっていた。でも、AI を使えば、1 時間で完了します」

「それは大きな効果ですね」

湊は画面を見つめたまま感心した。

「でも、AI が提案したテストケースをそのまま使うのではなく、人間がレビューして、必要に応じて修正する必要があります。AI は完璧ではないから」

高橋が少し考えてから補足した。

「そうですね。AI と人間の協働が重要ですね」

自動テスト作成の効率も大幅に向上した。AIが提案したテストケースをベースに、人間がレビューして修正する。そのプロセスで、テストケースの品質も向上した。

高橋は満足そうに言う。

「AIって、QAの仕事を奪うんじゃなくて、強化してくれるんですね。これまで手動でやっていた作業を自動化できるようになって、もっと創造的な仕事に集中できるようになりました」

開発チームとQA部の連携も強化された。AIが生成したテストケースを共有することで、開発チームとQA部の認識のずれが減った。

翌週のチームミーティングで、高橋は成果を報告した。同じ会議室に、湊や山田らが顔を揃えている。

「テスト自動化の実験が成功しました。開発チームとの連携も強化され、認識のずれが減りました」

山田が満足そうに頷く。

湊は高橋の報告を聞きながら、小さな成功を実感していた。AI活用の第一歩が成功した。でも、これで終わりではない。もっと戦略的にAIを活用する必要がある。

技術的負債検出の実験と失敗

テスト自動化の成功を受けて、湊は翌週のミーティングで次のステップを提案した。

「次は、技術的負債の検出を試してみませんか？ 山田さんと一緒に実験してみたいんです」

「手を動かしてみないと、どこまで拾えるかは見えないからな。まず試そう」

山田が頷く。

翌週、湊と山田は技術的負債検出の実験を始めた。二人でコードベースをAIツールに読み込ませ、分析結果を待った。

湊はAIツールにコードベースを分析させた。勤怠管理モジュール、経費精算モジュール、プロジェクト管理モジュール。それぞれのコードをAIに読み込ませ、問題箇所を特定してもらう。

数時間後、AIは分析結果を返してきた。

「この関数、複雑度高すぎますね。循環的複雑度が15を超えています。リファクタリングを検討してください」

「ここ、テストカバレッジ不足です。この関数にはテストがありません」

「このコード、重複が3箇所で見つかりました。共通化を検討してください」

湊は結果を見て、少しがっかりした。期待していたほどではなかった。

「思ったほど精度が高くないですね。表面的な指摘ばかりで、実際の問題を捉えきれていない気がします」

湊は山田に相談した。

「例えば、このモジュール間の依存関係について、AIは何も指摘していません。でも、実際には勤怠管理モジュールと経費精算モジュールの間で、データの整合性を保つための複雑な依存関係があるんです。それが問題の根本原因なのに、AIはそれを理解できていない」

山田は画面を見つめながら、深く考え込んだ。

「AIに仕事を任せるのは、新人に仕事を任せるのと同じだよ。仕様書もテストもない状態で『このコード直して』って言ったら、新人だって困るだろう？ 読めるものがないければ、的外れな答えしか返せない」

「なるほど……」

「このチームは以前よりはよくなったとはいえ、ドメインモデルが曖昧だったりテストが不十分なため、AIの理解が追いつかず、出力結果がいまいちな状態になっている。まずはコードベースや仕様を、AIが読める形で整えないといけない」

「具体的には、どんな問題があるんですか？」

「まず、ドメインモデルの境界が曖昧だ。勤怠管理モジュールと経費精算モジュールの間の依存関係が、コードからだけでは理解できない。AIはコードの表面だけを見て判断しているから、実際の問題を捉えきれない」

「それから、テストが不十分だ。テストがないと、AIはコードの意図を推測できない。この関数が何をしているのか、なぜこの実装になったのか、AIにはわからない。だから、表面的な指摘しかできない」

「それから、仕様や設計の情報が、AIがそのまま読める形でまとまっていない。仕様書や設計書を、コードと同じリポジトリでバージョン管理し、変更の履歴も残す。コードとドキュメントを同じ運用に乗せないと、AIは参照しにくい」

山田が続ける。

「身近な例だとロボット掃除機に近い。掃除機を回すには、先に床の物をどけて通れる道を作る。AIを本気で使おうとすると、コードや仕様を読みやすい形に整えたい。リファクタリングしたくなるのは、その表れだ。人が掃除機のために部屋を片付けるのと同じで、AIを効かせるには人間が先に土台を用意する」

湊は頷きながら聞いた。

「なるほど。AIをうまく使うには、まずシステム自体をAIリーダブルな状態にする必要があるんですね」

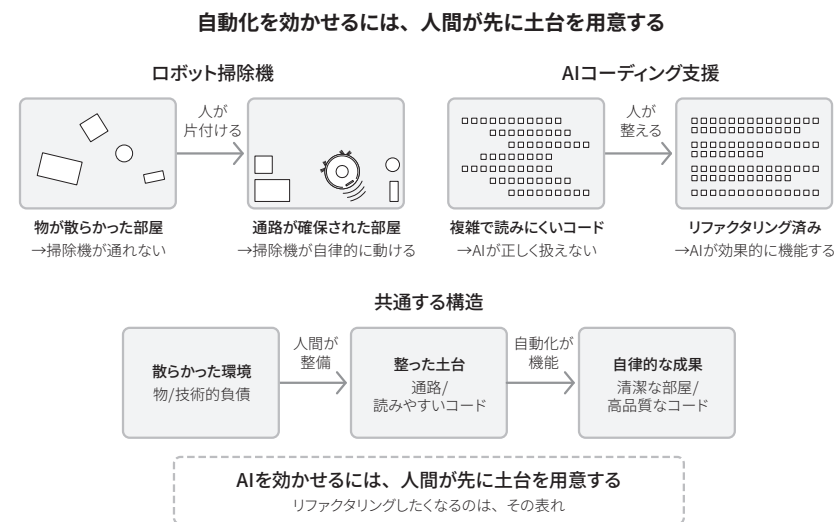


図5.2：自動化を効かせるには人間が先に土台を用意する。ロボット掃除機とAIコーディング支援の対比と共通構造

「そうだ。コンテキストエンジニアリングは、AIに適切な文脈を提供する技術だ。コードだけではなく、ドメインモデル、テスト、仕様書、設計ドキュメント。これらをAIが理解できる形式で整備することが重要だ」

コンテキストエンジニアリングへの取り組み

湊とチームは、コンテキストエンジニアリングの重要性を理解した。そこで、これまでの改善の延長として、コンテキストエンジニアリングに重点を置いた取り組みを始めることにした。最初の対象は、実験で課題がはっきりした勤怠管理と経費精算モジュールに絞った。

まず、その範囲の仕様書や設計ドキュメントをMarkdownで記述し、Gitでコードと同じリポジトリに置いてバージョン管理する体制を整えた。チームとAIが同じ参照元を見られるよう、ドキュメントをコードと同じ流れで更新する

仕組みを構築した。

次に、ドメインモデリングの改善に取り組んだ。イベントストーミングを活用して、ドメインモデルの境界を明確化した。勤怠管理モジュールと経費精算モジュールの間の依存関係を、イベントストーミングで可視化し、ドメインモデルの境界を明確にした。

そして、テストの充実を進めた。テストカバレッジを向上させ、コードの意図を明確にした。対象モジュールの各関数にテストを追加し、なぜその実装になったのか、どんな制約があるのかを、テストコードで表現した。

1ヶ月後、当面の改善は一段落した。ドキュメントはコードと同じリポジトリで管理される形に整い、ドメインモデルの境界も整理されていた。ただし対象は勤怠管理と経費精算に絞った段階で、プロジェクト管理モジュールなどほかの領域は、まだ手を入れていない。

改善後の技術的負債検出の実験

湊は、改善を進めた勤怠管理と経費精算まわりのコードを、再度AIツールに分析させた。

今度は、AIの分析結果が大きく変わった。

「このモジュール間の依存関係が不明確です。ドメインモデルの境界が曖昧で、影響範囲の把握が困難です。イベントストーミングでのモデリングを検討してください」

「この関数にはテストがありません。テストがないため、変更の影響範囲を把握できません。この関数は認証チェックとキャッシュ機能を実装していますが、テストがないため、これらの機能が正しく動作しているか確認できません」

「このモジュールのコード、特定のメンバーが頻繁に修正しています。他のメンバーが触っていないため、知識が属人化している可能性があります。コードレ

ビューやペアプログラミングで知識を共有することを検討してください」

「使用しているライブラリのバージョンが2つ以上古くなっています。セキュリティパッチが適用されていない可能性があります。バージョンアップを検討してください」

湊は驚いた。AIが指摘した箇所は、確かに問題のあるコードで、以前よりもはるかに的確な指摘になっている。

「改善後は、AIの指摘が的確になった。コンテキストエンジニアリングの重要性を実感した」

湊は興奮を抑えきれず、山田に報告した。

「予想以上の精度で問題箇所を特定できました。特に、モジュール間の依存関係や、テストがないことによる影響範囲の把握の困難さなど、以前は指摘できなかった問題を、正確に指摘できるようになりました」

山田も画面を見つめながら頷く。

「確かに、これは有用だね。コンテキストエンジニアリングを進めたことで、AIがシステムを正確に理解できるようになった。でも、AIが指摘した全てが問題とは限らない。人間の判断も必要だよ」

「そうですね。AIの指摘を参考にしつつ、人間が最終判断をする。それが重要ですね」

湊は結果をチームで共有した。全員が驚いた。

「AIって、こんなことができるんですね。しかも、システムを改善することで、AIの精度も上がるんですね」

佐藤が画面を指さしながら、感心した様子で言う。

「そうだ。AIをうまく使うには、まずシステム自体をAIリーダブルな状態にする必要がある。それがコンテキストエンジニアリングだ」

山田がホワイトボードを指差しながら補足した。

失敗から学ぶ

すべてがうまくいくわけではない。AI活用にも失敗はあった。

ドキュメント自動生成の実験では、期待した結果が得られなかった。湊はコードベース全体をAIに読み込ませ、APIドキュメントの自動生成を試した。

AIは30分で100ページのドキュメントを生成した。でも、湊が読み返してみると、内容が薄い。湊のデスクで、二人は生成されたドキュメントを開いていた。

「内容が薄い。AIが生成したドキュメントは、表面的な説明しかしていない」

湊が作成したドキュメントを見て、山田が指摘した。

「例えば、この関数の説明。『ユーザー情報を取得する関数です』としか書いてない。でも、実際には、この関数は認証チェックも行っているし、キャッシュも使っている。そういう重要な情報が抜けている」

山田は画面を指差しながら続ける。

「コンテキストが不足している。AIはコードの意図を完全に理解できないから、詳細な説明ができない。コードだけを見ても、なぜその実装になったのか、どんな制約があるのかはわからない」

湊は頷きながら聞いた。確かにAIが生成したドキュメントは、表面的な説明しかしていなかった。

コードレビュー支援の実験でも、問題があった。

湊は、AIにコードレビューを依頼した。AIは100行のコードに対して、50箇所の問題を指摘した。でも、佐藤が確認してみると、実際には問題があるのは10箇所だけだった。

「false positiveが多い。AIが指摘した問題の半分以上は、実際には問題じゃない」

佐藤が資料を広げて報告した。

「例えば、AIが『この変数名は不適切です』と指摘したけど、実際にはプロジェクトの命名規則に従っている。`user\_id`という変数名は、プロジェクトでは標準的な命名規則なんです。でも、AIは一般的な命名規則と比較して、不適切だと判断した」

佐藤は別の例も示した。

「それから、AIが『この関数は長すぎます』と指摘したけど、実際にはこの関数は複雑なビジネスロジックを実装していて、分割すると可読性が下がる。AIはコードの長さだけを見て判断しているけど、コンテキストを理解していない」

湊は深く考え込んだ。AIの限界と、人間の判断の必要性が頭を巡る。

「AIも万能じゃないんですね。コンテキストを理解できないから、誤検出が多い」

「でも、使い方次第で強力なパートナーになる」

山田が穏やかに続ける。

「ドキュメント自動生成は、完全に自動化するのではなく、AIが下書きを作って、人間が修正する。AIが生成したドキュメントをベースに、人間がコンテキストを追加する。そうすれば、効率的にドキュメントを作成できる」

「コードレビュー支援も、AIが指摘した箇所を人間が確認する。AIが指摘した箇所を、人間がフィルタリングして、本当に問題がある箇所だけを修正する。そう

すれば、効果的に使えるよ」

湊は失敗から学んだ。AI を完全に自動化するのではなく、AI と人間が協働する。AI が得意なこと（データ分析、パターン検出）を AI に任せ、人間が得意なこと（コンテキスト理解、意思決定）を人間が担う。それが重要だった。

対応荒れとロードタイム

AI エージェントの活用が進むにつれ、湊には別の負荷がのしかかっていた。複数のタスクを Agent に振ると、次々と「できました」と報告が上ってくる。そのたびに、湊はどのプロジェクトの何の文脈だったかを思い出し、成果物を開いて検収する。Agent からの質問や確認が Slack やツールに頻繁に飛んできて、一日中それに対応していると、プロジェクトについて深く考えたり、次のアイデアを検討したりする時間がなくなっていった。

ある日、山田が湊の様子に気づいて声をかけた。

「湊君、最近ずっと対応に追われてるみたいだね。昔の PM が『つらみ』で潰れてくのと同一構造だよ」

湊は苦笑いした。

「Agent がたくさん動いてくれるのはありがたいんですが、自分が指示を出して、出てきた成果を検収するのが追いつかなくて…。どのタスクの話か思い出して、文脈を組み立て直すのに毎回時間がかかってしまうんです」

山田は頷いた。

「それは『ロードタイム』って呼ぶんだ。プロジェクトを思い出して、頭のなかで文脈を組み立て直してから検収できるまでにかかる時間。これが3秒なのか3分なのかで、開発生産性は桁違いに変わる」

湊は聞き逃すまいと身を乗り出した。

「では、どうすれば短くできるでしょうか？」

「三つ、工夫がある。一つは、メモからアイデア、アイデアからタスクへの流れを対話的にして、思いついたらすぐタスク化して Agent に渡すこと。二つ目は、納品から検収、検収からリリースまでを、最小の認知コストでレビューできるフローにすること。三つ目は、どのプロジェクトにどれだけ Agent 時間を投下しているか、どこが詰まっているかを可視化すること。そうすると、対応荒れを防げる」

湊はメモを取りながら頷いた。

「ロードタイムを短くする仕組みと、タスクから検収までの流れの可視化ですね。やってみます」

それから数週間、湊はチームでメモをタスクに落とすフローを整え、検収のチェックリストを簡素化した。どのプロジェクトが詰まっているかをボードで見えるようにし、優先して検収するようにした。すると、対応に振り回されず、「考える仕事」に集中できる時間が少しずつ戻ってきた。

ボトルネックが移動し、レビュー待ちと設計待ちが増える

AI 活用が進む中、チーム内で新たな課題が浮上していた。

湊は個人の実装速度が上がったことを実感していた。AI とペアプログラミングをすることで、以前は10時間かかっていた作業が4時間で完了するようになった。でも、チーム全体のアウトプットが伸びているかというと、そうでもない気がする。

湊は実装を早く終えても、コードレビュー待ちで次の作業に進めないことが増えていた。レビュー依頼を出しても、レビュー担当者が対応できるまでに時間がかかる。以前は実装に時間がかかっていたので、レビュー待ちの時間は相対的に短く感じていた。でも、実装速度が上がった今、レビュー待ちの時間が相対的に長く感じられるようになった。

チームミーティングで、湊は状況を共有した。

「AI 活用で個人の実装速度は上がったんですが、チーム全体のアウトプットが思ったほど伸びていない気がします。実装を早く終えても、レビュー待ちで次の作業に進めないことが増えています」

山田が腕を組み、深く頷く。

山田がコーヒークップを傾けながら言った。

「前のチームでも同じことがあった。全員に AI ツールを入れて、個人の実装速度は確かに上がった。でも 3 カ月後に何が起きたと思う？」

「レビュー待ちが増えた……とか？」

「そう。プルリクの数が増えたのに、レビューする人数は変わらない。書いたコードがマージされるまでの待ち時間が伸びて、みんなイライラし始めた。速く書けるようになったのに、チームとしては前より遅い気がする。そんな状態だった」

山田の話聞きながら、湊はある事例を思い出した。AI が 3 分で書いたコードのリリースに 2 ヶ月かかったことがある。詰まっていたのはパイプラインではなく、人と人のあいだの見えない壁だった。

湊は息を呑んだ。

「では、何をレビューすればいいのでしょうか。コードを読まないなら、品質はどう保つのか」

山田は少し考えてから答えた。

「そこが、これからプロセスを変えるポイントだと思う。コードそのものではなく、『何を実現したいか』という意図や、仕様・制約・受け入れ基準をレビューする。コードはその成果物だとみなす。検証の基準も、コードを書く前の仕様の段階で決めておく。そうすれば、人間は機械より多く読むのではなく、上流で深く

考える側に回れる」

湊は理解した。

「つまり、実装速度が上がっても、レビューや設計のプロセスが変わらなければ、チーム全体の開発生産性は上がらないということですね」

「その通りだ。AI は個人を強化するが、同時にチームの人数を実質的に増やす効果も持つ。つまり、コミュニケーションラインという新たなボトルネックを顕在化させる。レビュー依頼が増え、レビュー担当者の負荷が増大する。設計の合意形成にも時間がかかる。これが、個人の実装速度が上がってもチームのアウトプットが伸びない理由だ」

山田はコーヒーを一口飲んでから続けた。「作れる量が増えたからといって闇雲に機能を足すと、使われない機能が増えて失敗する速度も上がる、という話を聞く。何を作るかを決める方が大事なんだよね。うまくいかなかったら捨てる前提にしておかないと、作れるだけ作るほどしんどくなる」

さらに、別の課題も浮上していた。

経験の浅いエンジニアが、**AI が生成したコードの意味を理解しないまま提出**し、レビューに余計な時間がかかるという問題だった。

佐藤が俯くように湊に報告した。

「湊さん、AI が生成したコードをそのまま提出したら、レビューで『なぜこのコードにしたの?』と聞かれて答えられませんでした」

速く出した分だけ、中身を説明する負荷がのしかかっている。理解を飛ばして書いたコードは、あとから読み返しても他人のコードのように感じる。湊はその感覚を思い返していた。

山田はホワイトボードに目をやってから説明した。

「AIが作ったコードへの説明責任は重要だ。経験の浅いエンジニアが、AIが生成したコードの意味や設計思想を十分に理解しないまま提出してしまうと、レビュー担当者からの質問に答えることができず、結果としてレビューに余計な時間がかかったり、本人の成長を妨げたりする危険性がある」

湊は状況を整理していた。個人の実装速度が上がったことで、レビュー依頼の数が増えている。以前は1日あたり2-3件だったレビュー依頼が、今は5-6件になっている。でも、レビュー担当者の数は変わらない。結果として、レビュー待ちの時間が長くなっている。

山田も同じ問題を感じていた。量が多すぎて、一つひとつなぜこうなっているかまで追いきれない、と呟いた。承認はしたが、中身を十分に理解したとは言い切れないものも増えている、という。

「レビュー依頼が増えているのに、レビュー担当者の数は変わらない。これがコミュニケーションラインのボトルネックだ。AIが個人を強化する一方で、チームの人数を実質的に増やす効果を持つ。つまり、コミュニケーションコストが増大し、意思決定プロセスがボトルネックになる」

湊は理解した。

「設計の合意形成でも同じことが起きています。実装が早く進むようになったので、設計レビューの頻度が増えています。でも、合意形成のプロセスは変わっていないので、意思決定に時間がかかっています」

湊は頷く。

「AIを使うときは、生成されたコードの意味を理解してから提出する必要があるということですね」

山田はさらに重要な指摘をした。

「湊くん、さきほど話したように、AIは組織の文化や実力を映し出す『組織の鏡』であり、良いチームでは開発生産性をさらに加速させる一方で、課題を抱える

チームでは混乱を広げてしまう『増幅器』として機能します」

湊は深く頷いた。

「つまり、AIを使いこなすためには、ツールの操作技術だけでは不十分で、エンジニア自身のスキルがより重要になるということです」

「その通りだ。AIの提案の良し悪しを判断し、その意図を理解し、自らの言葉で説明できるだけの深い知識と経験が、これまで以上に求められる」

AI エージェントによって、エンジニアは基礎に立ち返る

湊は深く考え込んだ。AIエージェントを使う中で、すべてが一周回ってエンジニアの基礎が必要なことに気づいていた。

設計がよくないと、AIは間違った品質のコードを吐いてくる。ドメインモデルが曖昧なままAIにコード生成を依頼すると、AIはその曖昧さを増幅させたコードを生成する。逆に、設計が明確で、ドメインモデルが整理されていると、AIは適切なコードを生成する。

それから、人が設計をわかっていないと、AIが生成したコードをレビューすることもできない。AIが生成したコードが設計思想に沿っているか、ドメインモデルの境界を守っているか、それを判断するには、レビュー担当者自身が設計を理解している必要がある。

湊は山田に相談した。

「山田さん、AIエージェントを使うと、すべてが一周回ってエンジニアの基礎が必要なのがわかります。設計がよくないとAIは間違った品質のコードを吐いてくるし、人が設計をわかっていないとそれをレビューすることもできない。スピードは早くなっても、逆にどんどん負債が溜まっていく」

山田がうなずいて応じた。

「その通りだ。AI エージェントは、エンジニアの基礎スキルを増幅させる。基礎がしっかりしているエンジニアは、AI を効果的に使いこなせる。でも、基礎が不十分なエンジニアは、AI を使っても負債を増やすだけだ。つまり、チームにとって育成が大事なんだ」

湊は理解した。

「AI エージェントを導入するだけでは不十分で、エンジニアの基礎スキルを育成することが重要だということですね」

「そうだ。設計の基礎、ドメインモデリングの基礎、コードレビューの基礎。これらをチーム全体で育成することが、AI 時代の開発生産性向上の鍵になる」

湊は深く考え込んだ。AI ツールを導入しただけでは、根本的な問題は解決しないのではないか。個人の実装速度は上がったが、チーム全体のアウトプットが伸びない。それは、コードレビューや設計の合意形成・意思決定のプロセスが変わっていないからだ。

「山田さん、ツールの導入だけでは構造は変わらないということですね。個人の実装速度が上がっても、コードレビューや設計のプロセスがそのままなら、組織全体の開発生産性は頭打ちになる」

山田が頷きながら続けた。

「その通りだ。開発生産性を最大化したいなら、意思決定の単位を小さくし、権限を分散させる組織設計が不可欠だ。ツールの導入だけでは、構造は変わらない。コードレビューのプロセスを改善し、要件定義の合意形成プロセスを改善する必要がある」

湊は理解した。

「つまり、AI ツールを導入するだけでなく、組織の意思決定プロセスそのものを見直す必要があるということですね」

「そうだ。コードレビューでは、自動承認基準を設定して、レビュー時間を短縮する。設計では、権限を委譲して、意思決定を迅速化する。そうすることで、ボトルネックを解消できる」

ツールの導入だけでは構造は変わらない

そうした気づきを踏まえ、湊はチーム全体で AI スキルの標準化・育成に取り組み始めた。

一部の詳しい人だけでなく、チーム全員が AI を効果的に使いこなせるよう、標準化と育成を実施した。AI 活用のベストプラクティスを共有し、組織全体に浸透させた。

チームミーティングで、湊は資料を手に、メンバーを見渡してから口を開いた。

「AI の提案を全て確認するのではなく、重要な部分だけに絞る。AI が生成したコードは、必ず意味を理解してから提出する。AI に依存しすぎず、自分で考える力を維持する」

山田が資料に目を通してから補足した。

「AI は補助ツールであり、代替ではない。AI が提案したコードをそのまま使うのではなく、人間がレビューして修正する。それが重要だ」

湊は続けて、組織設計の改善について提案した。

「それから、ボトルネックを解消するために、コードレビューと設計のプロセスを改善したいと思います。まず、コードレビューでは、自動承認基準を設定します。例えば、テストカバレッジが 80% 以上で、Lint エラーがなく、既存のパターンに従っている場合は、自動承認とします。これにより、レビュー時間を短縮できます」

湊は山田の言葉を思い出しながら続けた。

「あわせて、レビューする対象を変えたいんです。500行のdiffを全部読むのではなく、仕様や制約、受け入れ基準をレビューする。コードはその成果物だとみなして、検証の基準はコードを書く前に仕様の段階で決めておく。エージェントにコードもテストも書かせると、問題がテスト設計に移動するだけなので、何を正しいとするかは人間が先に決める。そうすれば、『遅くレビューしてバグを見逃し、本番でデバッグする』ではなく、『速く出して、すべてを観測して、もっと速く戻す』形に近づけられると思います」

山田が頷く。

「良い提案だ。レビュー担当者の負荷を減らし、レビュー待ちの時間を短縮できる。それから、設計の合意形成プロセスも改善する必要がある」

湊がホワイトボードに線を引きながら続ける。

「設計では、権限を委譲します。小さな設計変更については、エンジニアとテックリードで合意形成し、部長の承認は不要とします。大きな設計変更のみ、部長の承認を必要とします。これにより、意思決定を迅速化できます」

山田が資料に目を落としてから頷く。

「確かに、小さな設計変更まで部長の承認を取る必要はないですね。エンジニアとテックリードで判断できる範囲を明確にすることで、意思決定が迅速化されます」

佐藤が安堵した様子で頷く。

「AI疲れを感じていたけど、ガイドラインが明確になれば、もっと効果的に使えるようになりそうです。それに、コードレビューと設計のプロセスが改善されれば、待ち時間も減りそうです」

その結果、AI活用の浸透率が大幅に向上した。チーム全体でAIを効果的に使いこなせるようになり、開発速度の向上につながった。さらに、コードレビューと設計のプロセス改善により、ボトルネックが解消され、チーム全体のアウトプットも向上した。

湊はその夜、一人で考えた。人間は機械より多く読むのではなく、上流で深く考える側に回る。コードを全部読まなくても、意図と仕様と検証基準をきちんと決めておけば、速く出して、観測して、問題があれば速く戻す。そのパラダイムに、チームは少しずつ移り始めている気がした。

解説 AI活用の実践と組織的課題

この節の論点：AI活用の実践と失敗、ボトルネックの移動、アムダールの法則に基づく組織スループットの上限、そしてコンテキストエンジニアリングや組織設計の見直しによる対応です。

🔍 ストーリーで描かれる「重圧」

本質より対症療法が続いているとき、AI活用の「実践と組織的課題」は重くのしかかります。湊のチームが経験した重圧も、この構造から生じています。

- **テスト自動化・技術的負債検出でのAI活用と失敗** ドキュメント自動生成やコードレビュー支援では期待した効果が出ず、失敗から学ぶ
- **AI疲れ** AIから次々と出てくる提案を逐一判断・検証し続けることによる精神的な負荷。AIが作ったコードへの説明責任、レビュー時間の増加
- **ボトルネックの移動** 個人の実装速度は上がっても、チーム全体のアウトプットが伸びない。コードレビューや設計の合意形成プロセスがボトルネックになっている
- **「AI+人間」のペアプログラミングが最も効果的** という発見。組織の鏡としてのAI、設計がよくないとAIは間違ったコードを吐くこと、育成の重要性への気づき

この重圧の背景には、**AI導入で個人が速くなってもボトルネックが移動し、組織設計や協働パターンを見直さないと成果が出ない構造**があります。実践と失敗を経て、効果的な協働と組織的課題への対応段階にあるのです。

🔍 アムダールの法則と開発生産性の上限

第3章では、投資と生産性統計のズレや、AIの利用と組織の成果のあいだの構造的ギャップに触れました。ここでは、そのギャップを説明する**一つの枠組**

みとしてアムダールの法則を整理します。

1967年にコンピュータアーキテクトのジーン・アムダールが、並列処理による性能向上の限界を論じたもので、もともとはハードウェアの並列化の文脈で提唱されました。

ある仕事のうち「並列化できる部分」だけをどれだけ速くしても、「**並列化できない部分**」の割合がそのまま残るため、全体としての短縮には理論的な上限がある、というものです。並列化できない部分が全体の1割なら、残り9割を

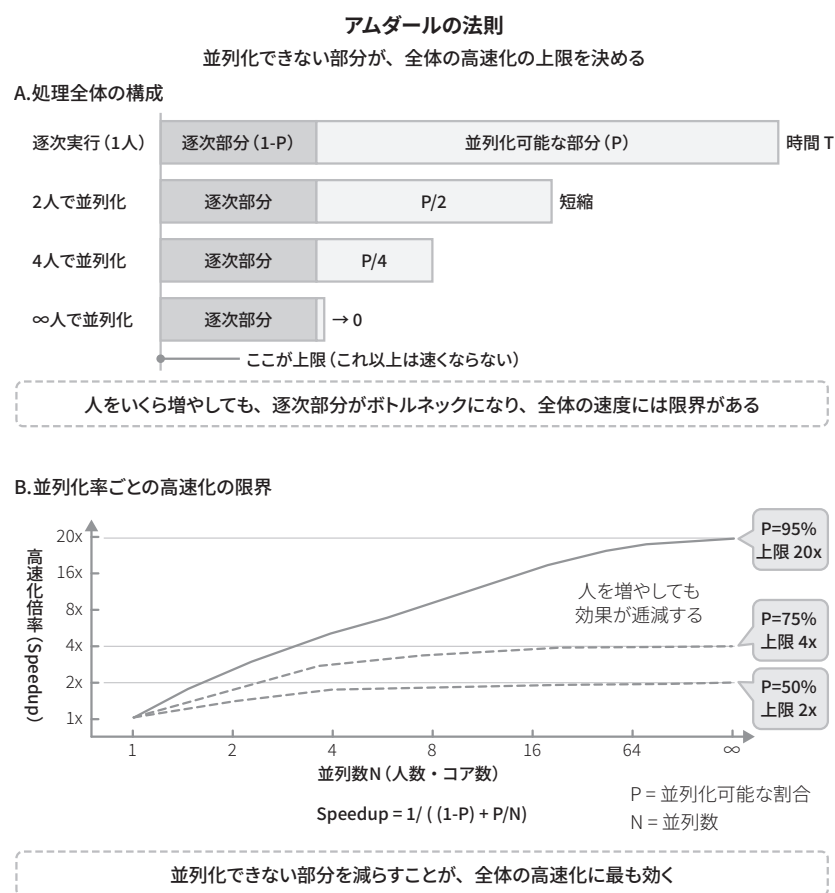


図5.3: アムダールの法則: 逐次部分と並列化可能部分の構成、並列化率ごとの高速化の限界と加速の式

無限に速くしても全体はせいぜい約10倍が上限です。並列化できない部分が3割なら上限は約3.3倍、5割なら2倍程度に抑えられます。つまり、**ボトルネックになる「直列部分」の割合が、どれだけ頑張っても超えられない天井**になるのです。

この図は、上段が処理全体の時間のうち**逐次部分（並列化できない部分）**と**並列化可能な部分**をどう分けるか、人数を増やしたときに後者だけが短くなる様子を示したものです。下段のグラフは、横軸を並列数N、縦軸を高速化倍率とし、並列化可能な割合Pを変えたときの加速曲線です。Pを50%・75%・95%とした例では、Nを増やすと加速しますが、**逐次部分は残るため、曲線の高いところで横ばいに近づき、天井があります**。天井の高さはPが大きいくほど高く(理論上Nを十分大きくすると、加速比はおおよそ $1/(1-P)$ に近づく)、人を増やしても効果が逓減する様子を読み取れます。図中の式は、加速比を $1 / ((1-P) + P/N)$ で表したものです。

技術的負債が蓄積すると、**レビューや影響範囲の把握に時間がかかり、実質的に「直列部分」が増えます**。その分、アムダールの法則でいう上限も下がり、一部の工程を効率化しても全体の伸びが限られることになります。

これを開発プロセスに当てはめると、次のように考えられます。並列化しやすく、AIで高速化しやすいのは、実装・コーディング・単体テスト生成・ドキュメント作成・環境構築などです。一方、顧客の課題の特定・設計判断・要件定

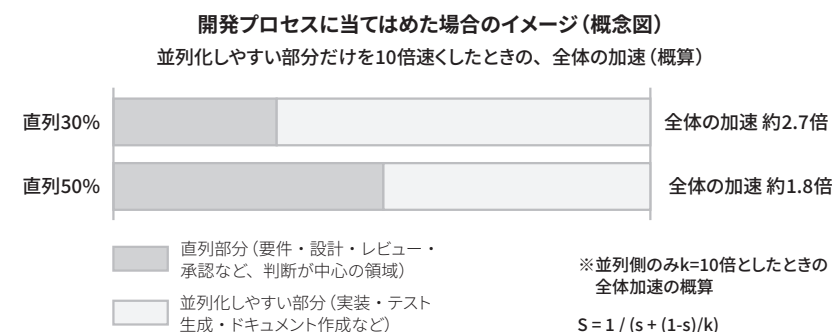


図5.4: 開発プロセスに当てはめた直列／並列の割合と、並列側のみ10倍速としたときの全体加速のイメージ (白黒)

義・レビュー・承認などは、人間の判断が中心で並列化しにくい領域です。本質的な仕事（直列部分）が全体の 30% を占める場合、それ以外を 10 倍速しても全体は約 2.7 倍にしかありません。50% を占める場合は約 1.8 倍程度です。

図の横棒は、左が直列部分（判断・合意・レビューなど）、右が並列化しやすい部分（実装・テスト生成・ドキュメント作成など）のイメージです。並列側だけを 10 倍速くした場合の全体の加速は、直列部分が 30% のとき約 2.7 倍、50% のとき約 1.8 倍となり、本文の数値例に対応します（式は図中の注どおり、並列側の加速を k 倍としたときの $S = 1 / (s + (1-s)/k)$ の概算です）。

だから「コーディングが速くなった」という体感があっても、組織全体のスループットは同じ倍率では伸びず、多くの組織で「AI で 2~3 倍で頭打ち」と感じる構造的な理由があります。「測れない」「数字に現れない」と感じる背景には、この法則で説明できる上限があるのです。

また、「AI を入れれば誰でも開発生産性が上がる」「ミクロの効率化がマクロの成果に波及する」といった通説は、実証研究では必ずしも支持されていません。California Management Review（2025）のメタ分析では、AI 導入と集計レベルの生産性向上の関係が一貫しないこと、高精度だが不完全な自動化により誤判断が増加しうることなどが報告されています。AI 導入＝開発生産性向上と短絡せず、湊のようにデータで可視化し、組織の適応（プロセス・レビュー・品質管理）とセットで考えることが重要です。

真の開発生産性向上は技術ではなく人材とプロセスの変革に 70% の投資を要する（BCG の 10-20-70）という示唆と整合します。

⑤ 認知的負債：AI 導入が生む見えない遅延

第 3 章で触れた認知的負債（コードは動くが、なぜそう動くかを組織的に理解できていない状態）は、AI 活用によって蓄積しやすくなります。「理解を飛ばして」実装できると開発速度は上がる一方で、理解の蓄積は人間の処理速度に縛られるため、出力と理解のギャップが認知的負債としてたまります。元になった英語記事では、古いコードほど危険になることや、オンコールが「黒箱を書いた黒箱」と向き合うこと、将来のシニア層が育ちにくくなることなど、AI 時代の失敗モードが cognitive debt という枠組みで整理されています。

認知的負債は客観指標でほぼ測れないため、組織は完了タスク数やマージ数などわかりやすい短期スピードにインセンティブが偏りやすく、認知的負債を増やす行動が評価されがちです。第 5 章で描かれたように、経験の浅いエンジニアが意味を理解しないまま AI 生成コードを提出すると、レビューキューには「理解のない出力」が溜まり、レビュアーは量に押されて十分に読み込めなくなります。「承認はしたが中身を十分に理解したとは言い切れない」レビューが増えると、「レビュー済み＝理解済み」という前提が崩れ、組織全体の認知的負債が加速していきます。

AI 活用の成果を測るとき、短期のスピード指標だけを見るとこの負債は見えません。「どれだけ速くコードが出たか」と同時に、「自分たちが説明できる範囲がどれだけ残っているか」という観点も意識することが、認知的負債を増やさないための補助線になります。

⑥ AI と人間の役割分担

湊と山田が発見したように、AI と人間が協働することで、より効果的な開発が可能になります。

AI が得意なこと（データ分析、パターン検出、網羅的チェック）を AI に任せ、人間が得意なこと（コンテキスト理解、創意工夫、意思決定）を人間が担う形にすると、開発全体の効率化につながります。

⑥ ボトルネックの移動

AI で個人の実装速度が上がっても、チームのアウトプットが伸びない理由は明快です。ボトルネックが移動するからです。この現象は、効率の捉え方の違いで整理できます。

リソース効率は、人や時間を無駄なく使うことを指します。人員の稼働率や稼働時間を最大化する視点です。コンビニのレジで従業員が常に忙しく動いている状態はリソース効率が高いといえますが、客の待ち時間は長くなりがちです。開発の現場では、個人の実装速度だけを上げると、レビュー待ちや承認待ちが増え、仕事の流れずチーム全体のアウトプットは伸びません。

これに対し**フロー効率**は、価値ある成果が早く届くことに焦点を当てた効率

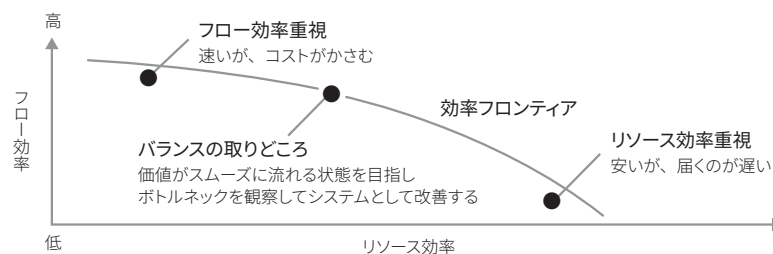
性です。アイデアから顧客に届くまでの時間を短くすることです。変化の速い環境では届ける速さが競争力になるため、多くのソフトウェア組織がアジャイルや開発と運用の一体化（DevOps）を採用するのは、フロー効率を重視するためです。待ち時間を減らす、バッチサイズを小さくする、流れのどこで詰まっているかを可視化するという行為が、フロー効率を高めます。

リソース効率だけを追うと届ける速さが落ち、フロー効率だけを追うとコストがかさむことがあります。両方を意識したバランスの取りどころを**効率フロンティア**と呼びます。チームの開発生産性を考えるときは、個人の稼働率ではなく、価値がスムーズに流れる状態を目指し、ボトルネックを観察してシステムとして改善する視点が必要です。

ソース効率とフロー効率



効率フロンティア



フロー効率を高める行動



図 5.5：リソース効率とフロー効率の比較、効率フロンティア、フロー効率を高める行動

一方、DORA（DevOps Research and Assessment）の 2025 年版レポート『State of AI-Assisted Software Development』では、AI エージェントの採用によりコード生成速度は上がった一方で**バッチサイズの増大**を招き、小さな変更を頻繁に出すという DevOps の原則が崩れ、レビュー負荷の増大やデプロイの不安定化が起きうると述べています。変更量の増加が許容される条件として、**制御システム**である自動テスト・成熟したバージョン管理・速いフィードバックを挙げ、これらがなくまま変更量だけが増えると不安定化する、という対比を示しています。AI は「入口（作成）」を広げるが、レビュー・テスト・リリース・運用のボトルネックを同時に解かない限り全体スループットは上がらない、という含意です。

Faros AI の「AI Productivity Paradox」(2025 年) にまとめられた分析では、企業の開発データ（テレメトリ）に基づき、**1,255 チーム・10,000 名超**の規模で観察されています。同じ報告については、第 4 章 4-1 の解説「消える開発生産性と「次の一手」の設計」でも、テレメトリの読み方と概念図つきで述べています。AI 高採用チームではタスク完了やマージ PR 数は増える一方、**PR レビュー時間が 91% 増**し人間承認がボトルネックになり、組織レベルでは AI 採用と DORA メトリクスの改善に有意な相関がなかったと報告されています。個人のタスク完了やマージが増えても組織の成果指標に結びつかない、という**開発生産性パラドックス**が、実データで示されたといえます。

個人の実装能力が 2 倍になっても、コードレビューや設計の際に必要な合意形成や意思決定のプロセスがそのままなら、組織全体の開発生産性は頭打ちになります。むしろ「待ち時間」の比率が相対的に増え、ストレスだけが増幅されます。

実装が速くなるほど、何を作るか・作らないかの意思決定がボトルネックになります。うまくいかなかったものを捨てる前提を組織で整えておかないと、作れるだけ作るほど負債が増えます。見積りの精度よりゴールの共有を重視し、リリース後の廃止や縮小を認める慣行にしておくことが、AI 時代の開発ではとくに効いてきます。

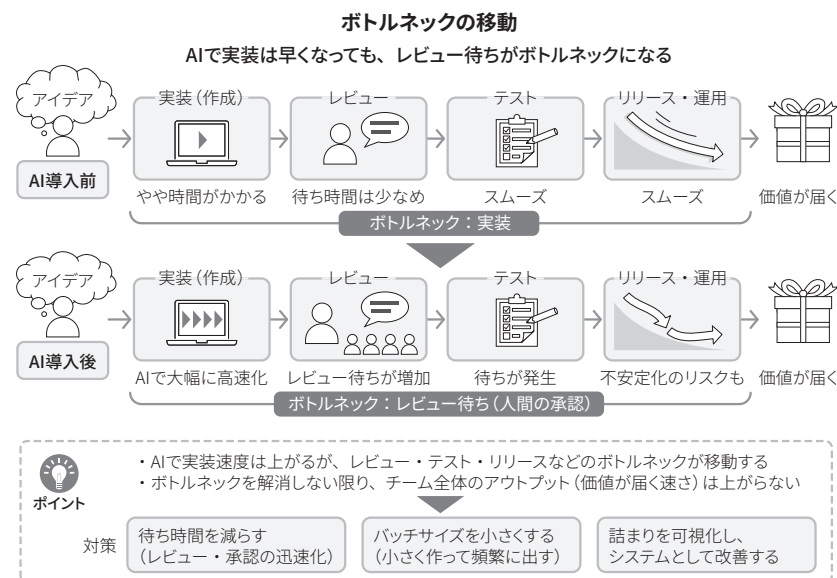


図5.6：ボトルネックの移動：AI導入前後の律速箇所の変化と対策（概念図）

⑤ 人間がボトルネックになる構造とロードタイム

AIエージェントと開発していると、本質的には「大量の入れ替わりをする部下」を抱えている状態になります。1つのAgentセッションは1タスクで終わり、次のタスクでは新しいAgentが起動する。このとき人間側に発生するボトルネックは三つあります。(1) **指示待ち**：Agentのタスクが枯渇して止まる。次に何をやらせるか人が考えないといけない。(2) **検収待ち**：Agentが「できました」と言っているが人がレビューできていない。成果物がキューに溜まる。(3) **対応荒れ**：Agentからの質問や確認が頻繁に飛んできて、人が常時対応する状態。特に三つ目は、日々の対応に追われてプロジェクトについて深く考えたりアイデアを検討したりする時間がなくなり、かつてのPMの「つらみ」と同型です。「考える仕事」は人間がやるしかないため、対応のコストを最小化して考える時間を確保する設計が必要です。

人間がボトルネックになる構造とロードタイム

AIエージェントは1タスクごとに入れ替わるため、人間側に次の3つのボトルネックが発生する

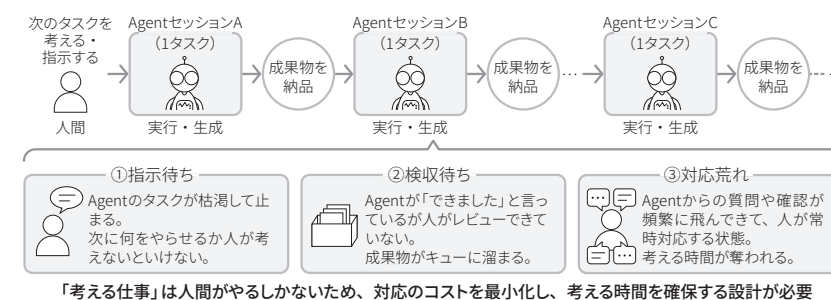


図5.7：人間がボトルネックになる構造とロードタイム：指示待ち・検収待ち・対応荒れ（概念図）

そのための鍵の一つが**ロードタイム**です。人がプロジェクトを想起し、メモリに展開し、考え始めるまでの時間。検収のロードタイムも同様で、「できました」と言われてから文脈を思い出し成果物を評価できる状態になるまでの時間。これが3秒なのか3分なのかで開発生産性は桁違いに変わります。メモからアイデア、そしてタスクへと進む対話、納品から検収、リリースへと進む対話を整え、どのプロジェクトにどれだけAgent時間を投下しているか、どこが詰まっているかをリソース可視化することで、ロードタイムを短くし、対応荒れを防ぐことができます。

⑤ AI疲れとコンテキストエンジニアリング

AIを導入したのに「かえって疲れた」と感じる現象を**AI疲れ (AI Fatigue)**といいます。AIは仕事の「密度」を高めます。同じ時間のなかで、意思決定や不確実性への対峙が増えるのです。意思決定は体力 (HP) ではなく**精神的なエネルギー (MP)**を消耗させますが、人間のMPはそう簡単には増えません。そのため、これまでと同じ内容の仕事を「AIで速く」こなそうとすると、限界が来ます。また、AIの自律性が低いと、AIが5分動くたびに確認と指示を挟む繰り返しになり、常に監視と判断が求められ、認知負荷が最大化します。

対処の一つが、**コンテキストエンジニアリング**です。AIに「いちいち聞かなくていい」状態を作る設計のことです。(1) **ルール化・明文化**：繰り返すフィードバックや判断基準をドキュメント化する。(2) **手順の明確化**：参照先や判断の優先順位を明示する。(3) **フィードバックループ**：戻れる仕組み (Issue、コ

メント)を整備する。これらを進めることで、AIを長時間(1~2時間)自律的に動かし、人間は最後に確認する形にできます。明文化が進むほど、人間は本質的な仕事に集中でき、認知負荷が下がります。湊のチームがコンテキストエンジニアリングに取り組むのは、AI疲れを防ぎ、真の開発生産性向上につなげるためです。

5-3

開発の投資区分による 戦略転換

ストーリー

作業時間が短縮されただけでは、真の開発生産性向上にはならない

これまでの取り組みで、湊とチームはコンテキストエンジニアリングまで進んでいた。AIがシステムを正確に理解できるようになり、技術的負債検出やテスト自動化などが可能になっていた。

しかしAI活用が進む中、湊はある疑問を抱くようになっていた。

確かにAIエージェントによってエンジニアの作業は楽になっている。でも、本当に開発生産性が上がっているのだろうか。作業時間が減った先に、何があるのか。

湊は具体的な例を思い浮かべた。以前は1つの作業に10時間かかっていたものが、AIの助けを借りて4時間でできるようになった。作業時間は確かに短縮されたが、残りの6時間をどう過ごすかはメンバーごとに違う。

あるメンバーは、その時間を使って新しい技術を学んでいる。別のメンバーは、より創造的な仕事に取り組んでいる。でも、中にはその時間を無駄に過ごしてし

まっているメンバーもいるかもしれない。

「AIで作業時間が短縮されただけでは、真の開発生産性向上にはならない。残りの時間をどう使うか、そういったことまで設計するように、きちんとした開発プロセスの再設計やチームのあり方を考えることが必要だと思う」

湊は深く考え込んだ。AIツールを導入するだけでは不十分だ。プロセスそのものを見直し、チーム全体が効果的に働ける仕組みを作る必要がある。

工数分析の過程

その夜、湊は一人オフィスに残っていた。フロアの照明はほとんど消え、自分のデスクだけが明るい。以前に学んだ手法を活用して、過去3カ月分の工数データを整理していた。

スプレッドシートにデータを入力しながら、湊はパターンを探していた。AIエージェントをどこに導入すれば効果的なのか。そのためには、まず現状の業務を整理する必要がある。

湊はスプレッドシートを見つめながら考え込んだ。数字の並びの向こうに、チームの時間の使い方が見えてくる。

人件費がある中で、Capacityとして毎月使える人月は固定化されている。その中で「新しい価値に使える工数」と「現在のシステムを運用する工数」で分けて考えることができるのではないかと。

湊は開発区分を整理してみた。

「新しい価値に使える工数」

- 新規開発：新しい価値（機能やサービス）を0から作る
- エンハンス開発：既存システムの拡張・変更

「現在のシステムを維持運用する工数」

- 保守開発：資産価値を耐久年数を維持する・伸ばす（振る舞いを大きく変えずに内部品質を整え、現状維持を行うリファクタリング・バージョンアップ等）
- 運用：障害対応、問い合わせ対応、ルーティンワーク等
- 管理業務、その他：会議、採用等の管理業務

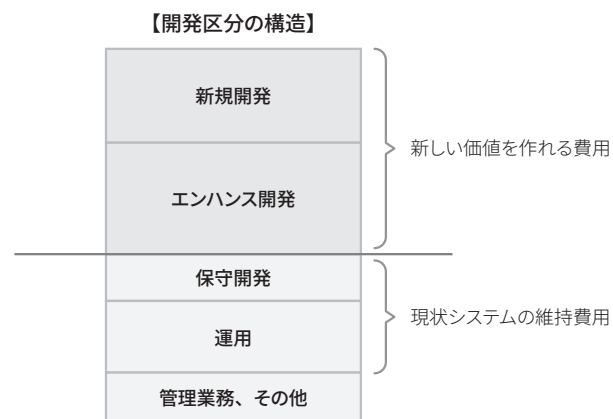


図 5.8：開発区分の構造：新規開発・エンハンス開発と保守開発・運用・管理業務、その他（新しい価値を作れる費用と現状システムの維持費用）

「これが開発の投資区分構造なのかもしれない」

湊は深く考え込んだ。この構造を理解することで、AI 戦略を整理できるのではないかな。

翌週のチームミーティングで、湊は開発投資区分の概念を提案した。ホワイトボードの前に立ち、メンバーの視線が集まる。

「これまでの AI 活用の成果を踏まえて、開発の投資区分の観点から AI 戦略を整理してみませんか？」

湊は先週整理した開発区分を、ホワイトボードに描きながら簡潔に説明した。

湊は説明を終えると、一息ついてから続ける。

「本来は『新しい価値に使える工数』に沢山使うべきですが、今は障害対応やバグ対応など『現在のシステムを運用する工数』に時間を使っているのではないのでしょうか」

湊は少し考え込むようにしてから続ける。

「PM や部長から見ると、エンジニアはみんなずっと開発をしているリソースに見えるかもしれませんが。しかし、実は運用工数が大きく占めているという誤解も、この区分で数値化することで理解が進むのではないかと思います」

山田が頷きながら応じた。

「なるほど。各区分を詳しく説明してもらって、構造がよく理解できました。これなら、どこに時間が使われているか一目瞭然だね。開発の投資区分という名前がぴったりだ」

飛鳥もホワイトボードに目を向け、興味深そうに見ていた。

「確かに、この構造で整理すると、現状の問題も明確になりますね。AI 戦略も考えやすくなりそうです」

湊は続ける。

「この開発の投資区分の観点から見ると、AI 戦略は 2 つに分けられます」

チームミーティングでの戦略整理

湊はホワイトボードに AI 戦略の 2 つの軸を描きながら説明を続ける。

戦略1：開発プロセスの高速化 (Increment を 1.5 倍にする)

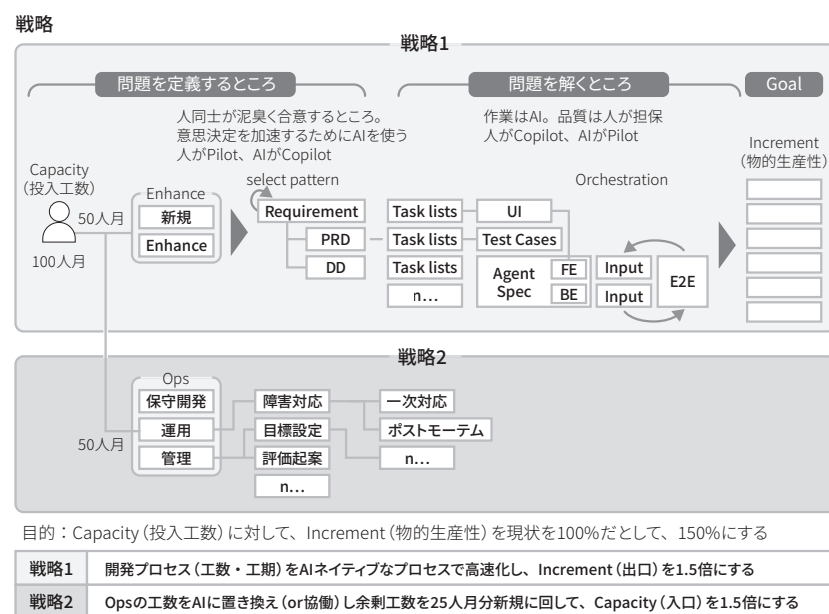


図5.9：開発の投資区分に基づくAI戦略：戦略1（Enhance）と戦略2（Ops）、Capacity・Increment、PilotとCopilotの役割分担のイメージ

湊はホワイトボードに「戦略1」と書きながら説明した。

「まず、開発プロセスの各工程を、AIネイティブな進め方で高速化することで、同じ投入工数でより多くの成果物を生み出す戦略です」

「問題を定義するところ」

湊は「問題を定義するところ」と書きながら続ける。

「要求定義をもとに要件定義を詰める部分で、AIを使って意思決定を加速しています。人がPilot、AIがCopilotという役割分担です」

飛鳥が少し考えてから頷く。

「確かに、要件定義の時間が短縮されて、意思決定が早くなりましたね。AIが情報を整理してくれるので、私たちは意思決定に集中できます」

「問題を解くところ」

湊は「問題を解くところ」と書きながら続ける。

「実装からリリースまでは、作業はAIが主導し、品質は人が担保します。人がCopilot、AIがPilotという役割分担です」

山田が図を指してから補足した。

「AIがタスクを分解し実装まで進めてくれるので、私たちは品質チェックに集中できます。これにより、実装からリリースまでの工程が大幅に高速化されました」

湊はまとめた。

「開発プロセス全体のリードタイム短縮により、Increment（出口）を1.5倍にすることができます」

戦略2：Ops工数のAI置き換え (Capacity を 1.5 倍にする)

湊は「戦略2」と書きながら説明を続ける。

「次に、Ops工数をAIに置き換えることで、新規開発に回せる工数を増やす戦略です」

工数分析のプロセス

湊は工数分析の結果を示した。

「開発区分における保守開発・運用・管理業務の工数を分析しました。全体の約40%を占めていました」

山田が補足した。

「その中で、AIによる改善インパクトがある部分をプロセスから探しました。例えば、ポストモーテムの作成や障害分析、目標設定の管理などです」

AI Agentの導入と効果

湊は具体的な AI Agent の導入例を説明した。

「まず、ポストモーテムを書く AI Agent を導入しました。障害対応後のポストモーテム作成が自動化され、工数が大幅に削減されました」

山田が少し間を置いて続ける。

「次に、障害分析 AI を導入しました。過去の障害パターンを分析し、事前に対策を提案してくれる AI です。これにより、障害対応の工数が削減されました」

湊は管理業務での AI 活用も説明した。

「メンバーの目標設定から評価までを AI がサポートし、管理業務の工数が削減されました。これまで手動で行っていた作業が自動化され、余剰工数が生まれました」

湊はまとめた。

「戦略 1 により Increment（出口）を 1.5 倍にし、戦略 2 により Capacity（入口）を 1.5 倍にすることで、全体として開発生産性を 150% に向上させることができます」

運用業務の AI 化

チームミーティングの後、湊は戦略 2 の具体化に取り組み始めた。議論で出た方針を、実際の業務に落とし込んでいく段階だ。

「戦略 2 を具体化するために、運用業務の効率化を進めましょう。単一プロセスの効率化ではなく、全体最適を考えて AI ネイティブなプロセスを設計する必要があります」

山田が補足した。

「その通りだ。既存のやり方を変えずに、無理やり AI ツールを当てはめようとすると、かえって手間が増えてしまう。プロセスを AI 前提に作り変える必要がある」

翌週、湊は運用業務の工数分析を進めた。過去 3 カ月分のデータを整理し、AI による改善インパクトがある部分を特定した。分析結果は、直前のミーティングで示した見立てと整合していた。

「開発区分における保守開発・運用・管理業務の工数を分析しました。全体の約 40% を占めていました。その中で、AI による改善インパクトがある部分をプロセスから探しました。例えば、ポストモーテムの作成や障害分析、目標設定の管理などです」

会議室にメンバーが顔を揃えた。チームミーティングで、湊は具体的な AI エージェントの導入例を説明した。

「まず、ポストモーテムを書く AI Agent を導入しました。障害対応後のポストモーテム作成が自動化され、工数が大幅に削減されました」

山田が続ける。

「次に、障害分析 AI を導入しました。過去の障害パターンを分析し、事前に対策を提案してくれる AI です。これにより、障害対応の工数が削減されました」

湊は管理業務での AI 活用も説明した。

「メンバーの目標設定から評価までを AI がサポートし、管理業務の工数が削減されました。これまで手動で行っていた作業が自動化され、余剰工数が生まれました」

1ヶ月後、運用業務の効率化により、余剰工数が新規開発に回せるようになった。

「Ops工数の削減により、余剰工数が新規開発に回せるようになりました。開発チーム全体が新規開発に集中できる環境が整いました」

湊はチームメンバーに語りかけた。

山田が満足そうに頷く。

「運用業務の効率化により、新規開発に集中できるようになりました。AIを戦略的に活用することで、開発生産性を向上させることができました」

開発の投資区分の観点からの整理

湊は開発の投資区分の図を指しながら説明を続ける。

「開発の投資区分の観点から見ると、AIへの投資が開発生産性の向上にどうつながるかが明確になります」

山田が図と湊の顔を交互に見て頷く。

「なるほど。AIへの投資が、Capacity（投入工数）とIncrement（成果物）の両方を改善することで、開発生産性を150%に向上させるということですね」

湊は続ける。

「はい。戦略1は、同じ投入工数でより多くの成果物を生み出すことで、Increment（出口）を1.5倍にします。戦略2は、Ops工数を削減して新規開発に回す工数を増やすことで、Capacity（入口）を1.5倍にします」

チームミーティングの後、5名チーム全員がAI戦略の方向性を理解していた。

湊はチームメンバーに語りかけた。

「開発の投資区分の観点から整理することで、AI戦略の効果を明確に説明できるようになりました。経営層にも理解してもらいやすい形で説明できると思います」

解説 開発の投資区分による戦略転換

この節の論点：開発の投資区分で工数を整理し、開発プロセスの高速化とOps工数のAI置き換えという2つの戦略に整理して取り組むことです。

⑤ ストーリーで描かれる「重圧」

戦略や評価の枠組みを組み替える段階で現れやすいのが、「単一プロセスの効率化の限界」という重圧です。湊のチームが経験したのも、ここからです。

- 単一プロセスの効率化だけでは真の開発生産性向上にならないという疑問。AIで作業時間が短縮されただけでは不十分だという気づき
- 湊が一人で工数分析を行い、開発の投資区分の観点を発見 新しい価値に使える工数と運用する工数の区別、CapacityとIncrementの関係
- 戦略1（開発プロセスの高速化でIncrementを1.5倍）と戦略2（Ops工数のAI置き換えでCapacityを1.5倍）の整理 チームで共有し、経営層に戦略の効果を説明しやすくなる手応え
- 開発の投資区分の観点からAI戦略の効果を明確に説明できるという実感

この重圧の背景には、局所最適から全体最適へ、開発の投資区分でAI戦略を整理しないと効果が説明できない構造があります。戦略転換を言語化し、説明責任を果たす段階にあるのです。

⑤ 開発の投資区分の観点からのAI戦略

湊のチームが実現したように、AIを戦略的に活用するには、開発の投資区分の観点から整理すると、戦略が明確になります。

BCGの調査（59か国・1,000名のCxO（最高経営責任者等）等）では、74%がAI活用から目に見える価値を十分示せておらず、全社横断で先進的能力を整え継続的に大きな価値を出しているのは4%のみです。AIリーダー企業は10-20-70ルール（10%アルゴリズム、20%技術・データ、70%人材・プロセス）

を実践し、実装課題の約70%は人材・プロセスに関すると言われています。真の開発生産性向上は技術ではなく人材とプロセスの変革に70%の投資を要するという示唆です。DORA 2025は、AI支援開発を「ツール導入」ではなく「**組織の仕事システムの変革**」と捉え、AIは増幅器であり既存の強み／弱みを増幅するとしています。社内プラットフォーム品質・ワークフローの明確さ・チームの連携といった基盤が、AIの便益を組織パフォーマンスへ接続する鍵だとしています。開発の投資区分で戦略を整理し、プロセスと人への投資を組み合わせることは、こうした知見と整合的です。

開発の投資区分の観点から、AIによる開発生産性向上の戦略を2つに分けられます。

目的 Capacity（投入工数）に対して、Increment（成果物）の量を現状100%から150%にする

戦略1：開発プロセスの高速化（Incrementを1.5倍にする）

- 「問題を定義するところ」：人がPilot、AIがCopilot（意思決定を加速）
- 「問題を解くところ」：人がCopilot、AIがPilot（実装からリリースまでを高速化）
- 効果：開発プロセス全体のリードタイム短縮により、Increment（出口）を1.5倍にする

戦略2：Ops工数のAI置き換え（Capacityを1.5倍にする）

- 工数分析から始め、AIによる改善インパクトがある部分をプロセスから探す
- 保守開発、運用、管理業務におけるAI Agentの導入
- 効果：Ops工数の削減により、余剰工数を新規開発に回し、Capacity（入口）を1.5倍にする

両戦略の組み合わせ 開発生産性を150%に向上させる

詳細な手法については章末のAIとの効果的協働ガイドのワークシートを参照してください。

5-4

成果と組織への影響

ストーリー

個とチームの開発生産性向上、両立への道のり

AIをチームの業務フローに組み込み、エージェント活用を本格化してから半年が経過した。湊のチームは着実に成果を上げていた。四半期をめぐるプレッシャーは続いていたが、この半年のあいだに計画を信じて立てられる土台が、以前よりはっきりしてきた。

チームミーティングで、湊は成果をまとめた。いつもの会議室に、メンバーが顔を揃えている。

湊はデータを見せながら続ける。

「AIエージェントの導入により、ポストモテム作成の時間が80%削減されました。障害分析AIにより、障害対応の工数が30%削減されました。管理業務のAI化により、管理業務の工数が50%削減されました」

山田がデータに目をやり、補足した。

「開発の投資区分の観点から見ると、戦略1のIncrement（出口）と戦略2のCapacity（入口）だとおもに戦略2に力を入れることで、全体として開発にさける工数が1.5倍になりました」

組織への影響

AI活用の成果は、開発チームだけでなく、組織全体に影響を与えていた。

湊は他部署へのヒアリングを始めた。開発チームの外から、どう見えているのか。それを確かめたかった。

まずは営業部を訪れた。

「開発チームの頑張りで、顧客満足度が上がってます。バグが減ったことで、お客様からのクレームも減りました」

営業部の担当者は笑顔で満足そうに言う。

「具体的には、どのくらい改善したんですか？」

「月に10件あったクレームが、今は3件程度になりました。お客様との関係も良くなってます」

次にサポート部を訪れた。オフィスの別フロアで、担当者が湊を迎えた。

「バグ報告が30%減って、すごく助かってる。以前は毎日のように緊急対応に追われてたけど、今は余裕を持って対応できるようになりました」

サポート部の担当者は感謝の言葉を述べた。

「開発チームの改善が、サポート業務にも良い影響を与えてるんですね」

「はい。バグが減ることで、サポート対応の時間も削減できました。その分、お客様への提案やフォローアップに時間を使えるようになりました」

マーケティング部も訪れた。

「新機能のリリースペースも安定してきた。以前は予定通りにリリースできなかったけど、今は計画通りに進んでいます」

マーケティング部の担当者は資料を手に、リリース計画の安定性を評価していた。

田中部長との対話

湊は他部署へのヒアリングを終えて、田中部長に報告した。部長室のドアを開け、資料を手に挨拶する。

「部長、AI活用の成果をまとめました。AIを活用する前に比べて開発できる工数が向上し、バグ検出率は40%向上しています。他部署からの評価も良好です」

田中部長は資料を見つめながら、時々頷く。しばらく読み進めてから顔を上げた。

「これは素晴らしい成果だね。経営層も注目しているよ。もしかしたら、来月の経営会議で成果発表の機会が設けられるかもしれない。その時は、湊君に発表してもらいたいと思っている」

湊は深く頷く。開発の投資区分の観点からAI戦略の効果を明確に説明できる。それに、他部署からの評価も得られている。経営層にも理解してもらえるはずだ。胸の奥に、小さな手応えが広がった。

解説 成果と組織への影響

AI活用の成果を数値化して示し、他部署・経営への波及して機能することを扱います。

⑤ ストーリーで描かれる「重圧」

前のシーンで湊のチームが経験した「AI活用の成果を組織に示す」という壁は、**成果の可視化と組織への波及の段階**を丁寧に進める必要があります。開発チームだけがAIエージェントを活用し、自分たちの開発を楽にするために利用するのではなく視野を広げていくことで成果を見える化していきます。

- **開発できる工数の向上、バグ検出率40%向上** 技術的負債返済の進行、チーム疲労度の軽減という定量的な成果

- **営業・サポート・マーケなど他部署からの評価** 顧客満足度の向上、バグ報告の削減、リリース計画の安定化。開発チームの改善が組織全体に影響している実感
- **田中部長から経営会議での成果発表の打診** 経営層も注目しているという言葉。次のステップへつなぐ可能性
- **「測れない価値」が「測れる価値」になった**という実感の共有

⑤ 段階的な成果

湊のチームが実現したように、AI活用の成果は開発チームだけでなく、組織全体に影響を与えます。

AI活用により、以下の成果が得られます。

- **開発速度の向上** AI+人間のペアプログラミングにより、開発速度が30%向上
- **バグ検出率の向上** AIによるテストケース生成により、バグ検出率が40%向上
- **技術的負債の返済** AIによる技術的負債検出により、技術的負債の返済が進む
- **チーム全体の疲労度軽減** 運用業務のAI化により、チーム全体の疲労度が軽減される

また、湊のチームのように、開発の投資区分の観点で効果を説明し、他部署評価や定量的成果で可視化することは、こうした「個人と組織のギャップ」を埋める取り組みになります。

- **営業部** 顧客満足度の向上、クレームの削減
- **サポート部** バグ報告の削減、緊急対応の削減
- **マーケティング部** リリース計画の安定化によるキャンペーンや広告費計画の安定化

これらの成果を経営層に説明することで、AIへの投資が開発生産性の向上にどうつながるかを明確に示すことができます。

手法5：AIとの効果的協働ガイド

AI活用の基本原則

AIを効果的に活用するためには、以下の原則を押さえます。

⚙️ 1. AIは補助ツールであり、代替ではない

- AIが提案したコードをそのまま使うのではなく、人間がレビューして修正する
- AIが指摘した問題をそのまま信じるのではなく、人間が確認する
- AIの判断を盲信せず、人間の判断を優先する

⚙️ 2. AIと人間の役割分担を明確にする

AIが得意なこと

- 大量のデータ分析
- パターン検出
- 網羅的なチェック

人間が得意なこと

- コンテキスト理解
- 創意工夫
- 意思決定

⚙️ 3. 段階的に導入する

- 小さな実験から始める
- 効果が確認できたら、段階的に拡大する
- 失敗から学び、改善を続ける

AI 活用の具体的な手法

⚙️ 技術的負債検出での AI 活用

手順

1. AI ツールにコードベースを分析させる
2. AI が指摘した問題箇所を人間が確認する
3. 優先順位をつけて、段階的に改善する

効果

- 網羅的な分析が可能
- 人間では見落としがちな問題を発見
- 時間の短縮

⚙️ テストケース生成での AI 活用

手順

1. AI に仕様書を読み込ませる
2. AI がテストケースを提案する
3. 人間がレビューして、必要に応じて修正する

効果

- 網羅的なテストケースの生成
- テストケース作成時間の短縮
- エッジケースの発見

⚙️ コードレビュー支援での AI 活用

手順

1. AI にコードを分析させる
2. AI が潜在的な問題を指摘する
3. 人間が確認して、実際の問題かどうかを判断する

効果

- レビュー時間の短縮

- 見落としがちな問題の発見
- コード品質の向上

AI 活用のチェックリスト

導入前

- ☒ AI 活用の目的を明確にした
- ☒ チーム全体で AI 活用の方針を合意した
- ☒ 小さな実験から始める計画を立てた

導入中

- ☒ AI が提案したコードを人間がレビューしている
- ☒ AI の判断を盲信せず、人間の判断を優先している
- ☒ 失敗から学び、改善を続けている

導入後

- ☒ AI 活用の効果を測定している
- ☒ チーム全体で AI 活用のスキルを向上させている
- ☒ 継続的に改善を続けている

AI ケーパビリティチェックリスト

AI 時代に成果を出し続ける組織が持つ 7 つのケーパビリティを、あなたの組織で確認してください。

1. 明確に周知された AI スタンス

- ☒ 経営層から AI 活用を支持する明確なメッセージが出ている
- ☒ 従業員が業務で AI を利用することを積極的に支援する方針がある
- ☒ AI 利用に関するガイドラインやポリシーが整備されている

2. 健全なデータエコシステム

- ☒ 社内データがサイロ化されず、統合されている

- ☑ データへのアクセスが容易で、高品質なデータ基盤がある
- ☑ データの民主化が推進されている

3. 小さなバッチでの作業

- ☑ 変更を小さく分割してリリースしている
- ☑ 迅速なフィードバックサイクルが回っている
- ☑ CI/CD パイプラインが整備されている

4. ユーザー中心の重点

- ☑ ユーザーの課題解決を目的とした開発が行われている
- ☑ 技術ありきではなく、ユーザー価値を起点に考えている
- ☑ ユーザーフィードバックを開発に反映する仕組みがある

5. AI の内部データへのアクセス

- ☑ AI ツールが社内ドキュメントや Slack の会話を理解できる環境がある
- ☑ 組織固有のコンテキストを AI が学習できる仕組みがある
- ☑ 情報共有と透明性が重視されている

6. 高品質な内部プラットフォーム

- ☑ 開発者が共通で利用できる高品質なツールやインフラ基盤がある
- ☑ プラットフォームエンジニアリングが推進されている
- ☑ 開発者体験（DevEx）が向上している

7. 強力なバージョン管理プラクティス

- ☑ 全ての変更が追跡されている
- ☑ 問題発生時に迅速に以前の状態へ復元できる
- ☑ トレーサビリティと再現性が確保されている

評価

- 7つ全て該当 AI 時代に成果を出し続ける組織の基盤が整っている
- 5-6個該当 良好な状態。残りのケーパビリティを強化する
- 3-4個該当 注意が必要。DevOps の基本原則から見直す
- 0-2個該当 AI 導入前に DevOps の成熟度を高める必要がある

出典 DORA（DevOps Research and Assessment）レポート 2025 年版
『State of AI-Assisted Software Development』

手法 6：プロンプトエンジニアリング事例集

効果的なプロンプトの書き方

AI を効果的に活用するためには、適切なプロンプトを書くことが求められます。

🔧 技術的負債検出のプロンプト例

以下のコードベースを分析して、技術的負債の可能性のある箇所を特定してください。

- ・循環的複雑度が 10 を超える関数
- ・テストカバレッジが 70% 未満の関数
- ・コードの重複が 3 箇所以上ある箇所
- ・命名が不適切な変数や関数

各問題箇所について、以下の情報を提供してください

1. 問題の種類
2. 問題の深刻度（高/中/低）
3. 改善の提案



⚙️ テストケース生成のプロンプト例

以下の仕様書を読んで、テストケースを生成してください。

機能：ユーザー登録

- ・ユーザー名：3文字以上20文字以下、英数字のみ
- ・パスワード：8文字以上、英数字と記号を含む
- ・メールアドレス：有効な形式であること

以下の種類のテストケースを生成してください

1. 正常系のテストケース
2. 異常系のテストケース（各入力項目のバリデーションエラー）
3. 境界値テスト（最小値、最大値、境界値-1、境界値+1）
4. エッジケース（特殊文字、空文字、null など）



⚙️ コードレビュー支援のプロンプト例

以下のコードをレビューして、潜在的な問題を指摘してください。

レビューの観点：

- ・コードの品質（可読性、保守性、パフォーマンス）
- ・セキュリティの問題
- ・バグの可能性
- ・ベストプラクティスへの準拠

各問題について、以下の情報を提供してください

1. 問題の種類
2. 問題の深刻度（高/中/低）
3. 問題の説明
4. 改善の提案



⚙️ プロンプトの改善方法

プロンプトを改善するには、以下のポイントを意識してください

- **具体的な指示を出す** 曖昧な指示ではなく、具体的な指示を出す
- **出力形式を指定する** AIがどのような形式で出力すべきかを指定する
- **コンテキストを提供する** AIが判断するために必要な情報を提供する
- **例を示す** 期待する出力の例を示すことで、AIの理解を助ける

人が足りないと言われたときの検証チェックリスト

「人が足りない」「エンジニアを増やしたい」と感じたとき、人数目標を決める前に、次の3軸で現状を検証してください。

戦略

- ☒ 何を、いつまでに作るかが明確になっているか
- ☒ プロダクトラインナップの拡大計画があるか（既存2〜3本の機能拡充だけか、新規が増えるか）
- ☒ 計画は誰が、どの根拠で作ったかが共有されているか

技術環境

- ☒ AIを前提にしたときの必要人数・役割を議論しているか
- ☒ 1年前と同じ前提で人数計画を立てていないか
- ☒ 実装以外のボトルネック（要件定義の遅れ、優先順位づけの不在）がないか

組織能力

- ☒ 要件を定義し、優先順位をつけ、「作らない」と決められる人・役割がいるか
- ☒ 技術責任者（CTO等）の負荷が適切に分散されているか
- ☒ 経営陣の間で「何をいつまでに作るか」「何人必要なのか」の期待値がすり合っているか

3つが噛み合っていないまま人数だけを増やしても、組織は「忙しいのに進まない」状態になりがちです。本当に足りないのは人数か、それとも「何を作るか」を決める力や知恵か、一度立ち止まって確認してください。

章のまとめ

本章では、湊のチームがAIを戦略的に活用し、持続可能な開発を実現する過程を描きました。第4章からの改善活動の延長として始まった重圧のなか、AI本格導入から半年ほどのあいだに開発速度の向上やバグ検出率の改善が進み、四半期のタイムリミットにも一定対応できる見通しが立ち、計画を信じて立てられる土台づくりの一助ともなった。

🔧 主な学び

1. **AIは「コードを早く書く道具」以上のパートナー** AIをコード生成だけに使うのではなく、開発全体を効率化するパートナーとして使う
2. **AIと人間の協働の重要性** AIを完全に自動化するのではなく、AIと人間が協働することで、より効果的な開発が可能になる
3. **失敗から学ぶ** すべてがうまくいくわけではない。失敗から学び、改善を続けることが重要。コンテキストエンジニアリングの重要性を理解し、システム自体をAIリーダブルな状態にする必要がある
4. **AI活用の組織的課題への対応** AI疲れ、説明責任、AIスキルの標準化・育成など、組織的な課題に対処することが重要
5. **開発の投資区分による戦略転換** 開発の投資区分の観点から、戦略1（開発プロセスの高速化）と戦略2（Ops工数のAI置き換え）を実践することで、開発生産性を150%に向上させる。単一プロセスの効率化ではなく、全体最適を考えてAIネイティブなプロセスを設計する重要性を学ぶ
6. **成果と組織への影響** AI活用の成果は開発チームだけでなく、組織全体に影響を与える。営業部、サポート部、マーケティング部など、他部署からの評価も得られる

🔧 次章への展望

本章でAI活用と持続可能な開発を描いた。次の章では、湊がエンジニアの視点だけでなく、PM、PdM、事業責任者の視点も取り入れた新しい開発生産性評価の枠組みを提案する過程を描く。

参考文献

本章でAI・開発リソース・組織的課題・戦略転換・成果を論じる際に参照した文献を、本文での言及順に挙げる。

- **Demirer et al.** 企業内ランダム化実験 (Copilot). Microsoft・Accenture・Fortune 100企業、計4,800名超。タスク完了数約26%増、ジュニアで最大40%近く、シニアは7~16%に留まる。
- **Peng et al.** Copilot初の大規模RCT。タスク完了速度55.8%向上（信頼区間21~89%）。
- **METR** (2025). "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity." ベテランOSS開発者16名・246タスク、Cursor + Claude等。AI使用群が平均19%遅く、主観と実際のパフォーマンスの乖離を示した。
- **GitClear**. "Coding on Copilot" 2024 Data Report / "AI Copilot Code Quality" 2025. 1.5~2億行規模。Churn倍増、リファクタリング減少、重複急増、中長期技術負債の加速。
- **DORA**. 2025 State of AI-Assisted Software Development. AIは増幅器。パッチサイズ増大、レビュー負荷、制御システムなしの変更増は不安定化。組織の仕事システムの変革として捉える。 <https://dora.dev/research/2025/dora-report/>
- **Faros AI** (2025). "AI Productivity Paradox." 1,255チーム・10,000名超。PRレビュー時間91%増、人間承認がボトルネック。組織レベルでAI採用とDORA改善に有意相関なし。
- **Forsgren, N. et al.** (2024). "DevEx in Action: A Study of Its Tangible Impacts." ACM Queue, Vol. 21, No. 6; *Communications of the ACM*, Vol. 67, No. 1. フィードバックループとチームレベルのアウトカム。
- **Ankit Jain** (2026). "How to Kill the Code Review." Latent.Space. レビュー対象をコードから意図（仕様・制約・受け入れ基準）へ移す、検証はコードより先に spec から定義する、信頼の多層化（スイスチーズモデル）、「速く出して観測して戻す」パラダイム。 <https://www.latent.space/p/reviews-dead>
- **Ranganathan & Ye**. "AI Doesn't Reduce Work—It Intensifies It." HBR. 83%が「AIが仕事量を増やした」、認知的疲労・バーンアウト（エントリーレベル61%等）。
- **JetBrains**. "AI Tool Switching Is Stealth Friction." 複数AIツールの使い分けが集中力を削ぐ指摘。
- **Qodo** (2025). "State of AI Code Quality in 2025." 品質と速度のトレードオフを克服しているチームは、AIによるチェック（テスト自動生成・レビュー）を組み込んでいる分析。
- **BCG** (2024). "Where's the Value in AI?" 59か国・1,000名超のCxO等。価値実現は4%のみ。10-20-70ルール（70%は人材・プロセス）。
- **McKinsey**. "Yes, you can measure developer productivity" 等。個人のタスク完了速度より、AIが生み出す複雑なシステムをどう統制するかが組織の開発生産性を分ける。
- **フレデリック・ブルックス**『人月の神話【新装版】』丸善出版、2014年。ブルックスの法則。本章で言及した急激な人員増加のリスクの根拠。

第 5 章 / 開発リソースは足りないのに増え続ける重圧

- **Rockoder.** "Cognitive Debt: When Velocity Exceeds Comprehension." Beyond the Code. AI 支援開発における認知的負債 (Cognitive Debt) を論じた記事。コード生成速度と理解速度の乖離、技術的負債とは別に「なぜそう動くかを誰も説明できない状態」が組織リスクとして蓄積すること、測定されないが評価インセンティブに強く影響することなどを整理している。<https://www.rockoder.com/beyondthecode/cognitive-debt-when-velocity-exceeds-comprehension/>